

Operational Realizations Below Random Systems

Unbounded and Efficient Machine Layers for Constructive Cryptography

Research draft - 29 July 2026

Abstract. Constructive Cryptography deliberately states its resource calculus above any particular computation model. The random-system layer therefore specifies interactive behavior without fixing how it is computed or which costs are efficient. We construct a lower stack in two stages. Finite typed single-token networks of probabilistic interactive Turing machines first give an unbounded operational image: persistent random tapes determine a strict first-visible-output history process. We then retain an exact vector ledger for work, peak space, randomness, traffic, activations, and ideal-resource access. Non-observable polynomial meters yield a finite bounded operational algebra, including cyclic connections; useful efficiency additionally requires separately quantified no-exhaustion, productivity, and realization witnesses. Uniformity is primarily one fixed code receiving the security parameter in unary. A concrete generated-network compiler proves that a fixed generator with one boundary and a polynomial aggregate envelope is only secondary syntax. Abstract ideal resources remain a charged specification sort. A public pre-sample reservation preserves each admitted kernel exactly, while unbounded responses use a metered streaming/reassembly interface. Contextual quotients give a proved finite costed system algebra and erasure homomorphism. On the route-safe subalgebra, a derived canonical-routing envelope makes physical plumbing behaviorally transparent while retaining its exact cost; maximal strict transcripts then give an injective homomorphism into a standard-Borel partial-random-system algebra. Converter absorption reindexes concrete profiles, so asymptotic nonexpansion is a corollary. Finally, a selected private processor, native-configuration store, named coin tape, and sequential lossless link refine the metered machine pathwise: transactional reservation preserves failed ledgers, a well-founded phase/microstep rank preserves progress, and a named projection preserves the native exact cost while retaining administrative overhead. This is a lower realization for Constructive Cryptography, not a Universal-Composability translation.

Status of the manuscript. The pathwise macro construction, exact ledger, finite-meter laws, workload-relative adequacy predicates, acyclic certificates including solved response-adaptive DAG invariants, affine-credit certificates, bounded-call oracle kernel, generated-to-fixed compiler, finite contextual system algebra, erasure homomorphism, profile-reindexed absorption theorem, route-safe partial-random-system embedding, and worked reductions are proved under their displayed hypotheses. The selected private transition-token processor, configuration store, coin tape, and sequential-link macro refinement are also proved. Comparison with another independently imposed carrier that treats nonresponse or feedback differently remains conditional. RAM, shared processors, visible clocks, leakage, reset, and general concurrency remain separate proposals. No claim about UC roles, corruption, session conventions, physical leakage, or a Lean formalization is made.

1 Introduction

1.1 Two directions through an abstraction hierarchy

A cryptographic theory can be developed in two directions. A bottom-up development first fixes machines, communication tapes, scheduling, a cost measure, and an efficiency class. It then defines protocols and security in that setting. A top-down development first identifies the algebraic operations and observations needed for cryptographic reasoning, proves theorems at that level, and instantiates lower levels only when an application requires them. Abstract Cryptography was proposed in the second spirit. Its explicit hierarchy places a general system algebra above discrete and random systems, and places implementations and cost models below them [1, Secs. 1.4-1.5]. Constructive Cryptography uses this hierarchy to describe protocols as constructions of resources, while retaining the composition theorem at the highest useful level [2, Sec. 4].

Top-down does not mean that computation has been denied. It means that a computation model is not fixed before one knows which facts should depend on it. A lower model is legitimate if its objects and operations satisfy the axioms used above. The higher theorem is then inherited. Conversely, a lower model can expose distinctions, such as time, memory, or locality, that the higher layer has intentionally forgotten.

The current discrete layer already describes rich interactive behavior. A deterministic discrete system is a partial history function. A probabilistic discrete system is a law over such functions. A random system is the observable transcript behavior of that law. None of these statements says how one machine step is taken, how a query is routed through a network, or why connecting two programs agrees with connecting their denotations. That is the missing boundary addressed here.

1.2 The problem is realization, not replacement

The aim is not to replace the resource calculus by Turing machines. It is to construct a map from a concrete operational algebra into that calculus:

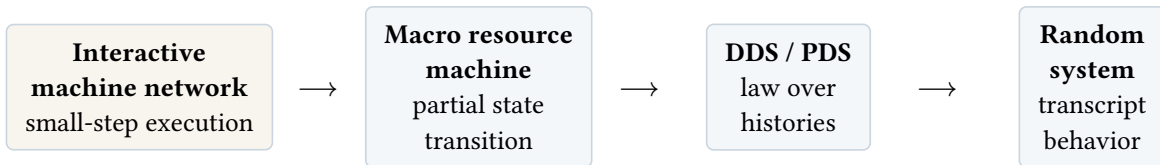


Figure 1: The abstraction ladder. Each arrow forgets internal information while preserving the observations used above it.

The first arrow forgets individual microsteps, head positions, and tape contents except insofar as they determine the next external response. The second forgets an explicit state space in favor of a canonical history function. The third forgets correlations that no one-shot transcript experiment can observe. The direction is therefore many-to-one at every stage.

The desired result is correspondingly not a representation theorem for every abstract system. It is a compositional realization of an operational image. That distinction is forced by computability: a perfectly valid mathematical history function can encode the halting set, while no ordinary Turing machine computes it.

1.3 Why the first implementation layer is unbounded

Information-theoretic security quantifies over computationally unrestricted tests. It does not follow that an operational account is meaningless. Three different questions are involved:

- Which mathematical behaviors are admitted as resources and tests?
- Which admitted behaviors arise from programs in a chosen machine model?
- Which costs of those programs are retained as observable resources?

We begin with behavior generated by finite probabilistic Turing-machine codes for the second question and erase time and tape consumption for the third. A terminating macrostep may take any finite number of microsteps and use any finite amount of tape. A machine may retain unbounded state between invocations. This instantiates the first information-theoretic regime in the modeling taxonomy of Maurer and Renner: converter

complexity is irrelevant at that abstraction [3, Sec. 3.5]. Within our operational image, every finite machine code is admitted without an efficiency restriction. This does not identify arbitrary noncomputable abstract converters with Turing machines, nor claim that memory is physically free. It declares only that implementation cost is not yet observable.

An operational layer remains useful in this regime. It validates that wiring is well-defined, identifies the finite-code machine-generated fragment, distinguishes nonresponse from a visible error, and supplies the lower implementation objects to which an abstract construction theorem can be applied. It also prepares later refinements: one can expose memory, processors, clocks, or communication delay as resources without changing the already proved higher theorem. The second half of this paper carries out the intervening refinement needed for computational security: it labels the same runs with exact costs, meters them, and takes a uniform polynomial family.

1.4 Contributions and scope

The paper makes nine contributions.

1. It defines a typed, single-token network model of probabilistic interactive multitape Turing machines. The model has persistent local state but no implicit scheduler, timeouts, shared randomness, broadcast, or fanout.
2. It defines a strict first-visible-output macro semantics and proves that a fixed tuple of random tapes induces a prefix-closed DDS. Product measure on named tapes then induces a measurable law over lifetime DDS behavior.
3. It isolates hypotheses under which denotation preserves parallel composition and connection. It states the resulting realization as a homomorphism into the random-system algebra and identifies the associated behavioral congruence.
4. It characterizes the boundary of the image through contextual soundness, finite-query full abstraction, finite representability, and explicit noncomputability counterexamples.
5. It refines the small-step semantics by an exact local and global cost ledger, then introduces non-observable component meters whose quotas are polynomial in a common ambient workload. It proves budget monotonicity, stabilization, and polynomial boundedness for fixed finite graphs, including cyclic ones.
6. It separates metered boundedness from no-exhaustion, productivity, and realization. Stop outcomes attribute the failed action to a public owner; context-safety, elementary and solved response-adaptive DAG certificates, and affine-credit certificates give nonvacuous availability rules.
7. It proves a fixed-boundary generated-network compiler and a finite cost-aware contextual system algebra. Erasure is a homomorphism to a behavioral operational quotient. On route-safe graphs, maximal strict transcripts give an injective homomorphism from that quotient into a standard-Borel partial-random-system algebra.
8. It separates finite-code implementations from charged abstract oracles, proves a pre-sample admission rule, streaming/reassembly coupling, and a concrete geometric-tail instance, and gives a budget-reindexed reduction calculus with complete RF/RP and persistent-mask calculations.
9. It fixes a private sequential processor/store/coin API and a lossless one-buffer link. A prepare/commit reservation protocol preserves success, block, exhaustion ownership, and the native exact ledger under a named projection. Its padded schedule has affine expansion, while the phase/microstep rank excludes new livelock; polynomial native grades therefore give polynomial overhead and preserve adequate efficiency.

The construction is classical and sequential at its external boundary. It is not a UC translation. UC and interactive-machine frameworks are mentioned only as operational evidence about reactive runtime. In particular, no UC-specific environment, adversary, corruption, session, or security-quantifier convention enters any definition below.

Section 2 introduces only the higher structures needed later. Section 3 fixes a running example. Section 4 and Section 5 define deterministic execution and its macro semantics. Section 6 adds probability. Section 7 and Section 8 state the realization and representability results. Section 9 and Section 10 separate productivity from cost and explain the role of memory. Section 11 adds exact costs; Section 12 develops bounded execution and adequacy, and Section 13 completes the route-safe partial-random-system map; Section 14 proves the efficient-family presentations; Section 15 and Section 16 connect ideal specifications, the finite costed algebra, and

computational distinguishers; Section 17 reifies selected costs as resources. Section 18 records the more general causal alternative, and Section 20 isolates the remaining carrier-comparison problem and later refinements.

2 Background and the target abstraction

2.1 Abstract reductions before cost models

It is useful to begin one level higher than resources. An abstract computational problem consists of a solver set Σ_P , an ordered performance set Ω_P , and a function

$$P : \Sigma_P \rightarrow \Omega_P. \quad (1)$$

An unconditional upper bound $P(s) \leq \varepsilon$ quantifies over every $s \in \Sigma_P$ and is independent of the implementation complexity of a solver. A reduction from a problem P to a problem Q consists abstractly of a solver transformation and a performance translation. A concrete computation model may later give a cost to those transformations, but the logical reduction does not require that choice. This separation is developed explicitly in [4, Sec. 4.4].

The same discipline applies to system construction. We first need to know what it means to connect systems, apply a converter, compare observations, and compose construction statements. Only then do we ask which lower machines realize the participating systems.

2.2 Resources, converters, and constructions

For the present purpose, a resource is a system with a finite family of labeled interfaces. A converter is a two-sided system with an outer interface and an inner interface. Attaching its inner interface to a compatible resource transforms the resource seen at the outer interface. Several resources can be placed in parallel, and compatible interfaces can be connected and hidden.

We assume an abstract class $\mathcal{R}$ of resources with:

- a typed parallel operation $\parallel$;
- a partial typed connection operation $\gamma_{p,q}$;
- interface renamings;
- identities represented by wires;
- a pseudometric or family of distinguishing functionals compatible with composition.

These are the system-algebra ingredients isolated by Matt, Maurer, Portmann, Renner, and Tackmann [5, Secs. 2-3]. Composition-order invariance says that a connected network depends on its typed connection graph rather than the order in which the expression describing that graph was built. It is a substantive property, not notation: a concrete lower model must prove it.

A constructive statement says, schematically, that applying a protocol to an assumed resource produces a system close to a desired resource, with a converter accounting for the remaining interface. The composition theorem then combines such steps. We need no further security syntax in this paper. Our task is prior to any particular construction claim: build a lower algebra and a homomorphism into $\mathcal{R}$.

2.3 Deterministic and probabilistic discrete systems

Let X be a query alphabet, Y an answer alphabet, and $X^+ = \bigcup_{n \geq 1} X^n$ the set of nonempty finite query histories.

Definition Deterministic discrete system (DDS). A deterministic discrete (X, Y) -system is a partial function $s : X^+ \rightarrow Y$ whose domain is prefix closed: whenever $x^n \in \text{dom}(s)$ and $1 \leq j \leq n$, the prefix x^j is in $\text{dom}(s)$.

The value $s(x^n)$ is the answer to the last query x_n after the preceding query history. Explicit state is absent because the history is a canonical sufficient state. This is the definition of Lanzenberger and Maurer [6, Def. 5], extending the input-output view of random systems in [7, Sec. 3]. Different programs, automata, or Turing machines can induce the same DDS.

Following [6], call such a DDS **finite** when X is finite and its domain is contained in $\cup_{i \leq n} X^i$ for some fixed n . Its graph is then finite. This bounded-horizon condition is stronger than merely having a finite-support law over possibly unbounded-lifetime DDSs.

A deterministic environment is a partial chooser $e : Y^* \rightarrow X$. Starting with $x_1 = e(\emptyset)$, it alternates $y_i = s(x^i)$ and $x_{i+1} = e(y^i)$ until either side stops. The resulting sequence of query-answer pairs is the transcript $\text{tr}(s, e)$.

Definition Probabilistic discrete system and transcript behavior. A probabilistic discrete system (PDS) is a probability law μ over DDSs. Its transcript law in a deterministic environment e is the pushforward of μ along $s \mapsto \text{tr}(s, e)$. Two PDSs are observationally equivalent if these transcript laws agree for every compatible deterministic environment. A random system is an observational equivalence class.

Lanzenberger and Maurer impose finiteness and a common DDS domain in their basic PDS definition [6, Defs. 8 and 10]. We will distinguish that finite common-domain theory from a measure-level extension needed for unbounded random tapes. In the extension, nonresponse is represented by absence from the DDS domain and not by inserting a visible error symbol.

2.4 Four presentations of one random system

The target development uses four adjacent mathematical views. They are worth separating because the operational layer enters before all four:

View	Carrier	Information retained
V1	$(\Omega, P, S : \Omega \rightarrow \text{DDS})$	A sample space, its probability law, and a random variable of deterministic lifetime behaviors.
V2	$\mu \in \mathcal{P}(\text{DDS})$	The pushforward law over DDSs; the names and internal structure of samples are forgotten.
V3	$p(y_i \mid x^i, y^{i-1})$	Conditional input-output kernels; counterfactual correlations invisible to one execution are forgotten.
V4	Transcript-law equivalence class	Exactly the laws observable through compatible interactive environments.

The passage V1 to V2 is pushforward. The passage V2 to V3 or V4 is an observational quotient. Maurer's original random systems use the conditional kernel view [7]; Lanzenberger and Maurer establish the law-over-DDS and transcript perspectives [6, Secs. 3.1-3.2]. A random automaton with explicit state and randomness is another presentation before V1; equivalent automata induce the same conditional laws [7, Sec. 3.1].

Our operational machine network supplies such a random automaton. Its small-step configuration is collapsed to a macro state, then to V1 by fixing random tapes. Nothing in the higher views can recover the number of machine steps, the tape layout, or the particular program.

2.5 Partiality and dependent interfaces

Partiality is essential. If $s(x^n)$ is undefined, the interaction has failed to produce the next answer. A context with no clock cannot distinguish a machine that halted silently from one that runs forever. Both are therefore absent from the DDS domain. A visible answer Err is different: the environment receives it and may continue.

For several typed interfaces, take a finite index set I , query sets X_i , and answer sets Y_i . The tagged query alphabet is $X_I = \sum_{i \in I} X_i$. Its answer family is dependent:

$$Y_{I(i,x)} = Y_i. \quad (2)$$

A dependent DDS maps each nonempty history ending in (i, x) , when defined, to an element of Y_i . This prevents a response at one interface from being mistaken for a response at another. The fixed-alphabet DDS is recovered when I is a singleton.

Notation	Meaning
$A = (Q_A, R_A)$	A call signature with request and response alphabets
$+A, -A$	Provider and client polarities
N, M	Operational machine networks
ω	A tuple of persistent component random tapes
δ_N^ω	The first-visible-output macro transition
$d_{N,\omega}$	The induced deterministic discrete system
μ_N	The pushforward law of $d_{N,\omega}$
$[N]$	The observational random-system denotation
$\parallel$	Disjoint parallel composition
$\gamma_{p,q}$	Connection and hiding of compatible ports

3 A persistent mask: the running example

Consider a resource K with one provider interface $\text{key} = (\text{Unit}, \text{bit})$ and a converter C with:

- an outer provider interface $\text{data} = (\text{bit}, \text{bit})$;
- an inner client interface for key .

The resource K reads its first random-tape bit k once, stores it, and returns k on every request. On an outer query x , converter C stores x , emits a request at its inner key port, waits for the response k , then returns $x \text{ xor } k$ and clears its pending input.

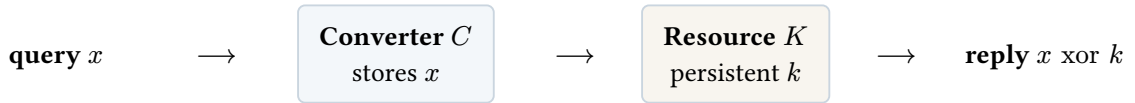


Figure 1: The persistent-mask example. The internal request and response are hidden after connection.

For a fixed bit k , the connected system is deterministic:

$$s_{k(x_1 \dots x_n)} = x_n \text{ xor } k. \quad (3)$$

For a uniform persistent k , its law over lifetime behaviors is

$$\mu_{C[K]} = \frac{1}{2} \delta_{s_0} + \frac{1}{2} \delta_{s_1}. \quad (4)$$

One answer is uniformly distributed. Two answers reveal a lifetime correlation:

$$(y_1 \text{ xor } x_1) = (y_2 \text{ xor } x_2) \quad \text{with probability 1.} \quad (5)$$

Resampling a fresh mask on each query has the same one-query distribution but a different two-query transcript law. This simple distinction will motivate three choices below: random tapes persist across macrosteps, a probabilistic system is a law over deterministic lifetime behaviors, and observational equivalence is determined by complete transcript laws rather than marginal answers.

4 Deterministic operational core

4.1 Call signatures and oriented ports

Definition Call signature. A call signature is a pair $A = (Q_A, R_A)$ of countable sets equipped with effective prefix-free encodings into finite bit strings. The countability and encodings are used only at the operational boundary; the macro semantics treats their elements abstractly.

An oriented port is a named occurrence of a signature with polarity $\varepsilon \in \{+, -\}$. A provider port $+A$ accepts requests and emits responses; a client port $-A$ emits requests and accepts responses:

$$\text{In}(+A) = Q_A, \quad \text{Out}(+A) = R_A, \quad \text{In}(-A) = R_A, \quad \text{Out}(-A) = Q_A. \quad (6)$$

A wire may connect only a $+A$ occurrence to a $-A$ occurrence. Thus an output of either endpoint is an input of the other. Every port occurrence has at most one wire. Broadcast, fanout, message dropping, reordering, and multiplexing must be implemented by an explicit component. This choice keeps connection structural rather than behavioral.

For dependent response types one replaces R_A by a family $R_{A(q)}$. Operational messages then carry an administrative request tag, and a well-returning machine must emit a response in the fibre of the active request. The tag is erased from the external transcript. All results below extend pointwise to this formulation; fixed R_A keeps the machine notation readable.

4.2 Interactive multitape machines

A component machine M consists of finite control, finitely many work tapes over finite alphabets, an input tape, an output register, and a one-way persistent random tape. Work tapes are two-way infinite but contain only finitely many nonblank cells in every finite configuration. A transition may:

- change the finite control;
- read and write the scanned work-tape cells;
- move each work-tape head by one position;
- either avoid the random tape or inspect the current random bit and, according to that bit's transition-table branch, stay or advance one cell;
- perform an internal step;
- emit one typed event at one of the machine's ports and suspend; or
- block without an event.

When an input event is delivered, its encoded payload is placed on the input tape and the machine enters its designated activation state. Work tapes, heads, finite control retained at suspension, and the random-tape position persist. The output register must decode to an element of the output alphabet of the selected port. We consider only codes satisfying this type condition.

Definition Component configuration. A configuration of M records its control state, finite-support work-tape contents, all head positions, the random-tape position, the input and output registers, and a mode in

$$\{\text{suspended, active, blocked}\}. \quad (7)$$

Once the infinite random tape $\omega_M \in \{0, 1\}^{\mathbb{N}}$ is fixed, the component transition is deterministic. “Probabilistic machine” will always mean this deterministic machine together with a measure on its random tape, rather than a primitive probabilistic transition.

The model is unbounded in a precise but ordinary sense. There is no uniform bound on the number of microsteps or visited tape cells. A successful observable response nevertheless occurs after a finite execution. Infinite execution is divergence, not a completed transfinite computation.

4.3 Networks and the single token

Definition Typed machine network. A network N consists of a finite set V_N of component occurrences, a finite set P_N of their pairwise distinct port occurrences, a partial matching W_N of compatible opposite-polarity ports, and the boundary ∂N of unmatched ports. Component names, port names, and random-tape names are globally distinct.

A global configuration is a tuple of component configurations together with one of:

- $\perp$, meaning that no component is active;
- an active component name $v \in V_N$;
- a terminal blocked marker.

A configuration with marker $\perp$ and all components suspended is **quiescent**. An external input may be delivered only to a quiescent network, at an unmatched boundary port p , with a payload in $\text{In}(p)$. Delivery activates the owner of p and creates the unique token.

While component v owns the token, an internal machine step changes only its local configuration. If v emits (p, z) , it suspends:

- if p is wired to p' , the event is hidden, z is delivered at p' , and the token transfers to the owner of p' ;
- if p is a boundary port, (p, z) is visible and the token disappears.

If the active component blocks, the global state becomes terminally blocked. There is no scheduler because at most one component is active. The state of every other component is passive data.

This is a deliberately sequential interface discipline. It still permits arbitrarily deep call chains and cycles. A suspended component may later be reactivated, and its tapes may encode a stack of pending calls. Dynamic process creation is unnecessary for expressiveness at this level: a finite universal machine can encode an unbounded collection of logical sessions on its work tapes. What the model does not express is genuine simultaneous activity or two unordered pending messages.

4.4 Runs and failure modes

For fixed network tapes $\omega = (\omega_v)_{v \in V_N}$ and a quiescent state c , delivery of a boundary input determines a unique maximal run. Four outcomes are possible:

1. a finite run reaches a first visible boundary output and a new quiescent state;
2. a finite run enters the blocked marker without a visible output;
3. infinitely many internal machine steps occur in one activation;
4. infinitely many hidden events transfer the token without a visible output.

The third outcome is **micro-divergence**. The fourth is **internal livelock**. At the abstraction boundary, outcomes 2-4 are all nonresponse. No clock is present with which to distinguish them.

Strict nonresponse. Nonresponse is not encoded as a response value. If an application needs a continuable rejection, it must include an explicit value such as `Err` in the response alphabet. Replacing blocking by `Err` changes the transcript: after `Err` an environment may issue another query, whereas after nonresponse no next round exists.

4.5 Structural operations

We identify networks up to bijective renaming of internal component, port, state, and tape names that fixes the external labels. The operations are:

- **Renaming.** A type-preserving boundary bijection $\rho : \partial N \rightarrow \partial N'$ changes only external names.
- **Parallel composition.** $N \parallel M$ is disjoint union after alpha-renaming all internal names. Its random-tape family is the disjoint union of the two families.
- **Connection.** If p and q are unmatched compatible ports of opposite polarity, $\gamma_{p,q}(N)$ adds the wire $\{p, q\}$ and hides the endpoints.
- **Feedback.** Connection is also allowed between two compatible boundary ports of the same network. We use “feedback” when emphasizing that the new wire may create a cycle.
- **Identity.** The identity on a signature is a stateless relay, or equivalently a bare typed wire in a presentation that permits wires as components.

These operations are defined on all well-typed finite networks. Connection need not preserve responsiveness: a feedback wire can create an infinite hidden run. Closure at the level of **partial** behavior is therefore easier than closure of a total or productive subclass.

5 Macro semantics

5.1 The first-visible-output transition

Fix a network N and one infinite random tape for every component. Let Q_N be the countable set of quiescent global configurations. Define the tagged boundary alphabets

$$X_N = \sum_{p \in \partial N} \text{In}(p), \quad Y_N = \sum_{p \in \partial N} \text{Out}(p). \quad (8)$$

An element (p, z) of X_N is an external input at port p . An element (q, w) of Y_N is a visible output at port q .

Definition Partial macro-transition. For fixed tapes ω , the macro-transition

$$\delta_N^\omega : Q_N \times X_N \rightarrow Q_N \times Y_N \quad (9)$$

is defined by

$$\delta_N^{\omega(c,x)} = (c', y) \quad (10)$$

exactly when the unique small-step run obtained by delivering x to c has a first visible event y after finitely many steps and the state after emitting y is the quiescent state c' . It is undefined if the run blocks, micro-diverges, or livelocks internally.

Because a fixed-tape network has one token and deterministic component transitions, there is at most one such pair (c', y) . The definition is strict: it does not search past a silent halt or an infinite internal run. The adjective “first” is redundant for a single successful macrostep but important for connection, where several hidden events may precede the visible one.

The transition forgets all intermediate configurations. It retains only the quiescent state needed to continue, the external input, and the first external output. In particular, two executions taking different numbers of microsteps can have the same macro-transition.

5.2 Folding macrosteps over histories

Choose an initial quiescent state c_0 in Q_N . For a nonempty input history $x^n = (x_1, \dots, x_n)$, define states and outputs recursively:

$$\delta_N^{\omega(c_{i-1}, x_i)} = (c_i, y_i), \quad 1 \leq i \leq n. \quad (11)$$

If every displayed transition is defined, set

$$d_{N,\omega}(x^n) = y_n. \quad (12)$$

If any transition is undefined, then $d_{N,\omega}(x^n)$ is undefined. Notice that the failed macrostep does not produce a successor state from which a later query could be evaluated.

Proposition Prefix closure. For every fixed-tape network (N, ω) , the partial function $d_{N,\omega} : X_N^+ \rightarrow Y_N$ has prefix-closed domain.

Proof. Suppose x^n is in the domain. By definition, there are states $c_0, \dots, c_n$ and outputs $y_1, \dots, y_n$ satisfying every transition equation through index n . For any $j \leq n$, the initial segment of the same witness satisfies the transition equations through j . Hence x^j is in the domain and $d_{N,\omega}(x^j) = y_j$. $\square$

Thus every fixed-tape network denotes a DDS. This proof is elementary, but it locates an important design choice. Prefix closure follows because an undefined response kills the current interaction. It would fail for a definition that silently skipped a failed round and resumed later.

Proposition Pathwise macro adequacy. Let x^n be a finite sequence of boundary inputs. The direct operational execution of (N, ω) produces visible outputs $y_1, \dots, y_n$, returning to quiescence after each output, if and only if x^n is in the domain of $d_{N, \omega}$ and $d_{N, \omega}(x^i) = y_i$ for every $i \leq n$.

Proof. Both sides unfold to the same sequence of macro-transition equations. The forward direction records the quiescent state after each visible output. The reverse direction concatenates the finite small-step witnesses in the definition of each macro-transition. Induction on n gives the claim. $\square$

5.3 Resource-shaped networks and dependent DDSs

The generic boundary alphabet permits an output at a port different from the one just activated. A resource interface normally has a stricter discipline.

Definition Resource-shaped network. A network is resource-shaped for interfaces $(Q_i, R_i)_{i \in I}$ if all boundary ports are providers and every successful macrostep begun by (i, q) emits its visible response at the same interface i . In the dependent variant, that response lies in $R_{i(q)}$.

For such a network the answer to a history ending in (i, q) lies in R_i , or in $R_{i(q)}$ in the dependent case. Consequently $d_{N, \omega}$ is a dependent multi-interface DDS. Return discipline may be enforced syntactically by a typed call stack, or semantically as an invariant of reachable configurations. This paper uses the semantic formulation because it accommodates universal machines whose stack is encoded on a work tape.

The persistent-mask network is resource-shaped at its outer data interface. For fixed k , its internal execution takes four visible-or-hidden event phases: deliver x to C ; route the hidden key request to K ; route the hidden response k back to C ; expose x xor k . Folding gives exactly Equation 3.

5.4 Converter-shaped execution

A converter has an outer provider port $+A$ and an inner client port $-B$. When activated by an outer request, it may either return an outer response or emit an inner request. After an inner answer is supplied, it again may make an inner request or return the outer answer.

For this shape, stop the small-step run at the next event of either kind. The result is a partial history function

$$\alpha : (Q_A^* \times (R_B \cup \{\perp\}))^* \rightarrow (Q_B + R_A), \quad (13)$$

subject to the evident alternation and length invariants. The left summand is the next inner query; the right summand is the completed outer answer. The $\perp$ entries record positions where the inner resource gave no answer in a total marked-history presentation; in the strict presentation, such a history has no successor.

Equation 13 is the operational origin of the partial protocol function used to define converter application at the random-system layer. Its state is not an extra semantic ingredient. Given fixed tapes and a reachable history, replay determines the suspended machine configuration. Keeping an explicit configuration merely makes execution efficient to describe.

5.5 The macro resource machine as an intermediate

There is a useful intermediate object between a network and its history function:

Definition Macro resource machine. The macro resource machine of (N, ω) has state set Q_N , initial state c_0 , and partial transition δ_N^ω . It has no microstep relation.

This object is exactly what one writes informally when specifying a stateful resource by an initialization procedure and a partial query handler. The operational realization developed here refines the handler into a network run; the DDS construction then folds that handler over histories. The factorization is therefore:

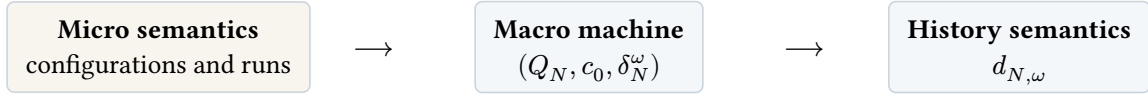


Figure 2: Macro-collapse separates operational adequacy from the canonical history representation.

The first arrow forgets step count and intermediate states. The second forgets the chosen state representation. Two macro machines with different state sets can still induce the same DDS, just as two programs can compute the same function.

6 Probability and lifetime behavior

6.1 Named persistent random tapes

For every component occurrence v , let

$$\Omega_v = \{0, 1\}^{\mathbb{N}} \quad (14)$$

with the product Borel sigma-algebra and fair Bernoulli measure P_v . The network sample space and measure are

$$\Omega_N = \prod_{v \in V_N} \Omega_v, \quad P_N = \prod_{v \in V_N} P_v. \quad (15)$$

The product is finite because the network graph is finite. Its coordinates are named by component occurrences. Alpha-renaming changes the names but preserves the product space up to the canonical measure-preserving bijection.

Independence is a modeling choice, not a consequence of the word “randomness.” If two components are meant to share coins, they must be wired to a common random source or otherwise receive a common seed. They must not silently read the same global tape coordinate.

The complete tuple ω is fixed for the lifetime of an execution. Each machine reads further bits as needed, but later macrosteps continue at the stored tape head. This is what turns probabilistic execution into a random choice of one deterministic lifetime behavior.

6.2 A measurable law over DDSs

Assume the boundary alphabets are countable. Give the set $\text{DDS}(X_N, Y_N)$ the sigma-algebra generated by the evaluation events

$$E_{h,y} = \{s : s(h) = y\}, \quad U_h = \{s : h \notin \text{dom}(s)\}, \quad (16)$$

for finite histories h and outputs y . Define

$$D_N : \Omega_N \rightarrow \text{DDS}(X_N, Y_N), \quad D_{N(\omega)} = d_{N,\omega}. \quad (17)$$

Proposition Measurability of fixed-tape denotation. If component transitions inspect only finitely many random bits in each finite small-step prefix, then D_N is measurable for the sigma-algebra in Equation 16.

Proof. Fix h and y . The event $D_{N(\omega)}(h) = y$ holds exactly when there exists a finite concatenation of successful small-step runs for the inputs in h whose last visible output is y . There are countably many such finite execution witnesses because configurations, event payloads, and finite traces are countable. Each witness constrains only the finitely many random bits read along it, hence describes a measurable cylinder subset of Ω_N . The inverse image of $E_{h,y}$ is their countable union.

The event that h is defined is the countable union over $y \in Y_N$ of these events. Therefore the inverse image of U_h , its complement, is also measurable. Since the sets in Equation 16 generate the DDS sigma-algebra, D_N is measurable. $\square$

We may consequently define the law over deterministic lifetime systems

$$\mu_N = (D_N)_* P_N. \quad (18)$$

This is V2 of Section 2. The original random-tape experiment (Ω_N, P_N, D_N) is V1. Conditional kernels or transcript equivalence produce V3 and V4.

6.3 Transcript laws and the observational quotient

For a deterministic environment e , the transcript map $s \mapsto \text{tr}(s, e)$ is measurable because every finite transcript event is determined by finitely many DDS evaluations. The network transcript law is

$$\text{Law}_{N(e)} = (\text{tr}(-, e))_* \mu_N. \quad (19)$$

Define $\mu \stackrel{\text{obs}}{=} \nu$ when these laws agree for every compatible deterministic environment. The random-system denotation is

$$[N] = [\mu_N]_{\stackrel{\text{obs}}{=}}. \quad (20)$$

This separates three semantic levels:

Level	Object	Equality criterion
Pathwise	$d_{N, \omega}$	The same answer or nonresponse for each fixed tape tuple and input history.
Probabilistic	μ_N	The same measure on deterministic lifetime behaviors.
Observational	$[N]$	The same transcript law in every compatible environment.

Pathwise equality implies equality of laws, and equality of laws implies observational equality. Neither converse holds in general. In particular, different couplings of answers to counterfactual first queries can give different laws over DDSs but the same one-execution transcript behavior. This is the distinction emphasized by the random-system quotient [6, Example 5 and Def. 10].

Proposition Probabilistic adequacy. For every deterministic environment e and finite transcript event A , the probability that direct random-tape execution of N in e produces a transcript in A equals

$$(\text{tr}(-, e))_* \mu_{N(A)}. \quad (21)$$

Proof. For each ω , pathwise macro adequacy identifies the direct operational transcript with $\text{tr}(D_{N(\omega)}, e)$. Taking the probability of the inverse image of A under these equal maps and applying the definition of pushforward twice gives the equality. $\square$

6.4 Why tapes persist

For the persistent-mask example, $D_{C[K]}$ takes two values, s_0 and s_1 , according to the first tape bit of K . Equation 18 therefore gives Equation 4. Equation 5 follows pathwise, before averaging.

If instead a semantic definition sampled a new deterministic system after every query, it would destroy this correlation. Such resampling does not describe one probabilistic interactive system; it describes a sequence of fresh systems. Fixing a complete tape first is also what makes eager sampling and lazy sampling candidates for observational equivalence: they may use different sample spaces and different counterfactual tables while inducing the same transcript laws.

6.5 Finite-support PDSs as a special case

The finite PDS theory is recovered under additional hypotheses.

Proposition Finite-image corollary. If $D_{N(\Omega_N)}$ is finite, then μ_N has finite support. If, in addition, X_N is finite and every DDS in this image has the same domain $D \subseteq \cup_{i \leq n} X_N^i$ for one finite n , then μ_N is a finite common-domain PDS in the sense of [6, Def. 8].

Proof. A pushforward is supported on the image of its map. Finiteness gives a finite-support law. The remaining hypotheses give both the common-domain and bounded-horizon finiteness conditions of the cited definition. $\square$

A machine whose only randomness is a finite initial seed has finite image and hence a finite-support lifetime law. It need not have a bounded interaction horizon. A bounded horizon together with a uniform bound on random bits read before that horizon yields a finite truncation in the cited sense. Neither property follows merely from absence of time and space bounds.

An infinite stream resource that returns the next fresh fair bit on every query has continuum many tape samples and infinitely many lifetime behaviors. Its law is not a finite distribution over DDSs. It nevertheless has perfectly well-defined finite transcript laws. Thus the full operational realization requires a measure-level PDS, or equivalently a compatible projective family of finite transcript laws. Finite-support lifetime laws remain exact special cases of this extension; when they also satisfy the finite common-domain hypotheses, the existing finite PDS presentation is recovered. Section 13 supplies the standard-Borel partial-DDS carrier and shows that this finite common-domain presentation embeds into it.

6.6 Random domains and termination conventions

Consider a machine that reads its first coin. On 0 it answers; on 1 it micro-diverges. The two fixed-tape DDSs have different domains. Its law is well-defined in the generalized DDS measure space, but it does not satisfy a literal common-domain support condition.

There are three principled responses:

- retain the generalized law with random-dependent DDS domains;
- restrict to networks whose domain is tape-independent, or equal almost surely to one fixed domain;
- work directly with transcript subprobability kernels.

Adding an error answer to force a common domain is not one of them, because it changes the observation available to an environment.

Sure termination and almost-sure termination must also be separated. A machine that flips until it sees 1 terminates with probability one but diverges on the all-zero tape. At the pathwise level its DDS is partial on that null set. At the measure level one may identify laws that differ only on a null set. A clean sure-terminating subcategory is possible, but it excludes useful exact samplers and is not necessary for information-theoretic transcript semantics.

7 Composition and operational realization

7.1 Hypotheses

The realization statements use the following hypotheses. Listing them separately prevents operational assumptions from being smuggled into the abstract system algebra.

ID	Hypothesis
H1	Networks and boundaries are finite; payload alphabets are countable and effectively prefix-free encoded.
H2	Ports are pairwise wired with matching signatures and opposite polarities; fixed-tape transitions and routing are deterministic.
H3	There is one activation token. Every successful macrostep ends at its first visible output in a quiescent state.
H4	Each component occurrence has a separately named random tape. Parallel components use the product measure unless sharing is represented explicitly.
H5	Nonresponse is strict. The abstract partial connection operation chooses the same least, first-visible-output feedback behavior as the operational semantics.
H6	The configuration and DDS spaces carry the standard Borel or evaluation sigma-algebras used in Section 6, and all structural maps are measurable.

ID	Hypothesis
H7	For dependent resources, reachable executions preserve the call and return typing invariant.

No time or space bound appears. No totality assumption appears. Additional productivity hypotheses will be introduced only when discussing a subcategory of responsive systems.

7.2 Renaming and parallel composition

Let ρ_*s denote the DDS obtained by relabeling every boundary event in a DDS s , and similarly for laws and random-system classes.

Theorem Renaming preservation. Under H1-H3 and H6,

$$[\rho N] = \rho_*[N]. \quad (22)$$

Proof. The renamed network has the same component configurations, random tapes, and microsteps. Only the tags of visible boundary events change. Therefore $D_{\rho N}(\omega) = \rho_*D_{N(\omega)}$ pathwise. Pushforward commutes with measurable function composition, so $\mu_{\rho N} = \rho_*\mu_N$. Passing to the observational quotient proves the claim. $\square$

For DDSs s on boundary B and t on disjoint boundary C , define $s \boxtimes t$ by interleaving tagged queries: a query to B advances only the s history, and a query to C advances only the t history. It is undefined exactly when the selected component DDS is undefined on its projected history.

Theorem Independent parallel preservation. Under H1-H4 and H6,

$$\mu_{N \parallel M} = (\boxtimes)_*(\mu_N \times \mu_M) \quad (23)$$

and hence

$$[N \parallel M] = [N] \parallel [M]. \quad (24)$$

Proof. Fix (ω_N, ω_M) . A boundary input to N changes no configuration or tape head of M , and conversely. The unique token therefore executes exactly the selected component network. Induction on the tagged input history gives

$$D_{N \parallel M}(\omega_N, \omega_M) = D_{N(\omega_N)} \boxtimes D_{M(\omega_M)}. \quad (25)$$

H4 identifies the tape law with $P_N \times P_M$. Applying pushforward gives the law equation. Transcript quotienting gives the random-system equation. $\square$

The independence condition is necessary. If both components read the same first global bit, their first answers are correlated, while the abstract parallel of independent laws is not. Shared coins must therefore be a named common resource rather than an accidental implementation convention.

7.3 Connection and feedback

Connection is the central nontrivial case because a boundary event becomes a hidden activation and can cause further hidden events. At the DDS level, $\gamma_{p,q}$ must be a strict feedback operation: beginning with an external query, repeatedly follow the uniquely determined hidden call-response chain until the first remaining boundary output; return undefined if the chain blocks or is infinite.

Conditional claim Deterministic connection correspondence. Assume H1-H3, H5, and H7. For every fixed tape tuple ω and every compatible pair of boundary ports p, q ,

$$D_{\gamma_{p,q}N}(\omega) = \gamma_{p,q}(D_{N(\omega)}). \quad (26)$$

Proof sketch. Relate a configuration of the physically connected network to the configuration used by the abstract strict-feedback evaluator by equality of every component configuration and ownership of the unique token. Internal microsteps match directly. An emission away from p, q is visible on both sides and ends the

macrostep. An emission at p or q is hidden on both sides, delivered to the opposite endpoint, and preserves the relation. A finite induction handles successful runs. For blocking and infinite runs, coinductive determinism shows that neither side can produce a first visible event. Folding the resulting macro-transition equality over histories gives Equation 26.

At this stage the formal obligation is to instantiate the abstract connection evaluator used by the target random-system algebra and verify that its least or strict fixed-point convention is exactly H5. Matching port types alone would not suffice. The maximal-transcript carrier and route-safe theorem in Section 13 discharge this obligation for the target selected in this paper. $\square$

Feedback is not a different semantic operation. It is connection whose new wire may lie on a cycle. Equation 26 still permits the result to be partial. It makes no claim that a previously total network remains total.

Conditional claim Probabilistic connection preservation. Assume H1-H7 and deterministic connection correspondence. Then

$$\mu_{\gamma_{p,q}N} = (\gamma_{p,q})_* \mu_N \quad (27)$$

and

$$[\gamma_{p,q}N] = \gamma_{p,q}[N]. \quad (28)$$

Proof. Connection neither creates components nor resamples tapes, so its sample space and measure are those of N . Equation 26 says that its denotation map is $\gamma_{p,q} \circ D_N$. Functoriality of pushforward yields the first equation. Compatibility of transcript equivalence with abstract connection yields the second. $\square$

The last compatibility is itself part of the target system-algebra interface. It is precisely why the observational quotient must be a congruence rather than an arbitrary equivalence relation.

7.4 Order independence

Let $e = \{p, q\}$ and $f = \{r, s\}$ be two legal disjoint wires. Adding e and then f produces the same finite matching as adding f and then e .

Proposition Structural order independence. Modulo alpha-renaming of internal names,

$$\gamma_e(\gamma_f(N)) = \gamma_f(\gamma_e(N)). \quad (29)$$

More generally, every ordering and parenthesization that builds the same finite typed network graph yields isomorphic operational networks.

Proof. Parallel composition is disjoint union modulo alpha-renaming, hence is associative and commutative up to the chosen network isomorphism. Connection is union of the current partial matching with a new two-element edge. Union of distinct legal edges is commutative and associative. Components, machine codes, and tape names are unchanged. The resulting graphs are therefore isomorphic, and the isomorphism preserves every small-step run. $\square$

Combining this proposition with connection preservation yields composition-order invariance of the denotational image. This is the concrete proof obligation highlighted by the abstract system-algebra program [5, Secs. 3-4]. Defining networks as graphs makes the structural half short, but does not remove the connection correspondence obligation.

7.5 Behavioral congruence and the image theorem

Define machine behavioral equivalence by

$$N \equiv_{\text{beh}} M \iff [N] = [M]. \quad (30)$$

Conditional claim Congruence and operational image. Under H1-H7 and the connection correspondence, $\equiv_{\text{beh}}$ is a congruence for renaming, independent parallel composition, and every well-typed connection. The quotient of machine networks by $\equiv_{\text{beh}}$ is isomorphic to the image of $N \mapsto [N]$, and that image is a subalgebra of random systems.

Proof. Preservation of each operation implies that replacing an operand by one with equal denotation leaves the denotation of the compound network equal. Thus the kernel relation of the denotation map is a congruence. The standard first isomorphism argument identifies the quotient by the kernel with the image. Closure of the image follows by applying each operational constructor and its preservation equation. $\square$

Theorem Compositional operational realization, conditional form. Subject to H1-H7 and the deterministic connection correspondence, the three-stage map

$$\text{machine networks} \rightarrow \text{laws over DDSs} \rightarrow \text{random systems} \quad (31)$$

is a homomorphism for renaming, independent parallel composition, connection, feedback with strict partial semantics, and identities. Its image, not the class of all random systems, is the operationally realizable subalgebra.

Proof. The identity relay induces the identity history function. Renaming and parallel preservation were proved above. Connection and feedback follow from the conditional correspondence and Equation 28. Structural order independence ensures that compound expressions depend only on the resulting typed graph. The congruence proposition supplies the quotient and subalgebra statement. $\square$

The theorem is conditional only at its genuine seam: matching the chosen abstract feedback operator with the small-step execution. It does not conceal a complexity or scheduler assumption.

The carrier selected later in Section 13 discharges this seam. Its strict connection is defined by the displayed hidden expansion, and its maximal-transcript equivalence is proved to be a connection congruence. The route-safety premise is needed only after physical routing receives a finite meter; the unbounded canonical router cannot introduce exhaustion.

8 Contexts, distinguishers, and representability

8.1 Operational contexts

An operational context $Z[-]$ is a finite machine network with one typed hole. Plugging a compatible network N into the hole means taking disjoint union, connecting the designated ports, and hiding them. A test context has one remaining decision port, and every successful decision response is a bit. It may be partial. The event $Z[N] = 1$ means that the finite run emits 1; nonresponse contributes no accepting mass and is not silently converted into a visible 0.

For a query budget q , let $\mathcal{Z}_q$ be a specified class of such contexts that cause at most q external resource queries before deciding. Define

$$d_{\text{Mach}}^{q(N,M)} = \sup_{Z \in \mathcal{Z}_q} |P[Z[N] = 1] - P[Z[M] = 1]|. \quad (32)$$

At the random-system layer, let $\mathcal{E}_q$ be the corresponding class of all abstract q -query environments, and define

$$d_{\text{RS}}^{q(S,T)} = \sup_{e \in \mathcal{E}_q} |P[e[S] = 1] - P[e[T] = 1]|. \quad (33)$$

Every operational context has a denotation, and composition preservation gives

$$[Z[N]] = [Z][[N]]. \quad (34)$$

The right side is schematic notation for plugging a denoted system into the denoted context.

Proposition Contextual soundness. If every context in $\mathcal{Z}_q$ denotes an environment in $\mathcal{E}_q$, then

$$d_{\text{Mach}}^{q(N,M)} \leq d_{\text{RS}}^{q([N],[M])}. \quad (35)$$

Proof. Probabilistic adequacy and composition preservation identify the acceptance probabilities of each operational test with those of its denoted abstract environment. The supremum defining d_{Mach}^q is therefore taken over a subset of the environments used for d_{RS}^q . $\square$

This is the direction needed to inherit an abstract security bound. It does not require every abstract environment to be implementable.

8.2 Image-relative full abstraction

Let $\mathcal{E}_{\text{op}}, q$ be exactly the denotations of contexts in $\mathcal{Z}_q$. Define $d_{\text{RS,op}}^q$ by taking the supremum only over this image.

Proposition Image-relative full abstraction. Under probabilistic adequacy and composition preservation,

$$d_{\text{Mach}}^{q(N,M)} = d_{\text{RS,op}}^{q([N],[M])}. \quad (36)$$

Proof. The two suprema contain the same tests up to denotation, and corresponding tests have equal acceptance probabilities. $\square$

Although formally immediate, this statement identifies the correct full-abstraction boundary. Operational equivalence is equality under operational contexts, while random-system equivalence quantifies over the chosen abstract context class. They coincide without qualification only when that class is generated by realizable contexts.

There is a useful exact finite-horizon corollary.

Proposition Finite-query exactness. Suppose X and Y are finite, the query bound q is fixed, and abstract environments are deterministic partial query choosers followed by a deterministic decision rule. Every such q -query environment is implementable by a finite-state machine. Consequently

$$d_{\text{Mach}}^q = d_{\text{RS}}^q. \quad (37)$$

If randomized environments are allowed, the same equality holds because their acceptance advantages are convex combinations of deterministic ones.

Proof. There are finitely many answer histories of length less than q . Store in finite control the current history, look up the next query or stop action in the environment's finite table, and update the state after each answer. The final decision table is implemented similarly. Thus every deterministic abstract environment lies in the operational image. A randomized environment is a distribution over deterministic tables; its signed acceptance difference is linear, so its absolute advantage is no larger than that of some deterministic extremal table. $\square$

The fixed bound is important. With countably infinite alphabets or an unbounded horizon, an arbitrary mathematical chooser table need not be computable.

8.3 Which systems are implementable?

Representability at the system side exhibits the same boundary.

Proposition Finite DDS implementation. Every finite DDS over effectively encoded finite alphabets is realized by a deterministic interactive Turing machine.

Proof. Hard-code the finite graph of the partial history function. The machine stores the input history on a work tape. On a new query it appends the encoding, searches the hard-coded table, emits the recorded answer when one exists, and blocks otherwise. Prefix closure ensures that every reachable proper prefix of a defined history has already produced its response. $\square$

Proposition Finite-support probabilistic implementation. Let $\mu = \sum_{i=1}^m a_i \delta_{s_i}$ be a finite-support PDS of finite DDSs.

- If every a_i is dyadic, one machine realizes μ using a bounded initial fair-bit sample.
- If every a_i is rational, one machine realizes the same transcript law using an almost-surely terminating rejection sampler.

Proof. For dyadic weights, choose r so that every a_i is an integer multiple of 2^{-r} . Read r bits, partition the 2^r strings into blocks of the required sizes, store the selected index, and emulate the lookup machine for s_i forever.

For rational weights, use a common denominator d . Choose r with $2^r \geq d$, sample an r -bit integer, reject values at least d , and repeat. A value below d is uniform; partition the d values according to the integer numerators. The sampler diverges only on a null set of tapes, after which the selected DDS is emulated. $\square$

The rational construction is exact at the level of transcript laws but not sure terminating on every tape. If a common-domain PDS is required pathwise, the null divergent tapes must be treated by an explicit almost-sure quotient or excluded by a stronger seed primitive.

The persistent-mask system is the smallest nontrivial instance of the dyadic construction: one bit selects between s_0 and s_1 .

8.4 Two failures of surjectivity

Example A noncomputable DDS. Let $X = \mathbb{N}$ and $Y = \{0, 1\}$. Define a total one-query DDS $s_{K(n)} = 1$ exactly when the n th Turing machine halts on empty input. This is a valid abstract DDS. A deterministic realization would decide the halting problem by direct execution. Random tapes do not evade the obstruction. For fixed n , the fair-tape sets on which a finite-code machine emits 0 or 1 after finitely many steps are effectively enumerable unions of dyadic cylinders, so both probabilities are lower semicomputable. Exact realization of the total deterministic DDS makes one probability 1 and the other 0. Dovetailing their lower approximations until one exceeds $\frac{1}{2}$ therefore decides $s_{K(n)}$, again a contradiction. Hence s_K is not in the machine-only operational image.

Example A nonrealizable probability. For every real $\alpha \in [0, 1]$, there is an abstract one-query random system that returns 1 with probability α . There are uncountably many such systems but only countably many finite Turing-machine descriptions. Hence some, and in particular some with noncomputable parameters, have no realization in the proposed model.

These counterexamples rule out a surjective realization theorem. Adding an oracle for every abstract DDS would make surjectivity true by definition but would cease to answer the computational question. The proper claim is a compositional embedding of a quotient of ordinary machines into the abstract algebra.

8.5 Lifting constructive statements

Suppose operational networks N_R , N_S , and P denote an assumed resource R , a target resource S , and a protocol denotation π , respectively. If the higher theory proves

$$d(\pi R, S) \leq \varepsilon, \quad (38)$$

and composition preservation identifies $[P[N_R]] = \pi R$, then every compatible machine test Z satisfies

$$|P[Z[P[N_R]] = 1] - P[Z[N_S] = 1]| \leq \varepsilon. \quad (39)$$

This is simply contextual soundness applied to Equation 38. It is the practical payoff of the lower layer: the abstract construction theorem is reused without rebuilding its quantifier structure inside the machine model.

9 Reactivity, productivity, and closure

9.1 Three notions that should not be conflated

The word “bounded” can refer to different properties:

- **Computation time** counts microsteps taken by one machine activation.
- **Interaction depth** counts hidden requests or token transfers before the next visible response.
- **Productivity** says that a visible response eventually occurs, surely or almost surely.

The unbounded behavioral model developed so far erases computation time. It does not automatically bound interaction depth, and its general denotation is partial because it does not assume productivity.

Some converter presentations require a uniform finite bound on the number of inner queries made before producing an outer answer. Jost uses such a condition in the discrete converter layer [8, Def. 2.2.2]. In the present setting that condition is best read as a reactivity requirement making converter application easy to close. It is not a polynomial-time or efficiency requirement: each of the finitely many activations may perform an arbitrarily long finite computation.

9.2 Local termination does not compose to productivity

Example Internal ping-pong. Let A have an internal client port $-U$ and an external provider port, and let B have the matching internal provider port $+U$. On an outer request, A takes one local step, emits a request to B , and suspends. On every request, B takes one local step, emits the corresponding response, and suspends. On every such response, A emits a fresh request instead of returning an outer answer.

Every component activation terminates after one step. Nevertheless the connected network transfers the token forever and never emits an external event. Its macro-transition is undefined by internal livelock.

Thus “every local call performs finite work” is not a compositional productivity condition. Nor does totality of isolated open components imply totality after feedback. A proof that connection preserves totality must use a global hypothesis.

9.3 Two closure strategies

The first strategy is the one adopted in the main model:

Partial closure. All well-typed finite networks are admitted. Connection always produces another network. A block or infinite internal run denotes undefinedness. The partial DDS category is closed even when a total subcategory is not.

The second strategy carves out a productive subcategory. Several sufficient conditions are available, with different expressive costs:

1. A fixed natural number bounds all hidden token transfers between two visible events.
2. Every reachable hidden transfer strictly decreases a global well-founded rank.
3. The connection graph is acyclic with respect to the possible call direction.
4. A guarded-feedback discipline ensures that each cycle crosses a visible boundary event before it can repeat.

The first condition is simple and resembles a bounded-inner-query converter discipline. The second is more flexible and supports data-dependent finite interaction depth. The third is syntactic but excludes useful recursion. The fourth is natural when the interface protocol exposes rounds.

For randomized networks, almost-sure versions of these conditions form a different theory. Almost-sure productivity may be preserved by some connections and destroyed by others; proving closure requires probabilistic ranking or martingale arguments. We do not assume such a theorem here.

9.4 No implicit timeout

A context that can count microsteps distinguishes an immediate response from one produced after 10^{100} steps and distinguishes some divergences by timeout. Such a context is not present in the random-system observation

model. Adding it while retaining Equation 20 would be unsound, because the macro-collapse has forgotten precisely that information.

Timing can be added later as an explicit clock or event coordinate. Until then, blocking, micro-divergence, and internal livelock are observationally the same nonresponse. This is not a claim about their physical equivalence; it is the declared observation boundary.

10 Memory and computation as resources

10.1 The first unbounded regime

The proposed machine layer gives every component persistent work tapes and places no bound on their use. It also gives every successful macrostep unbounded finite computation time. After macro-collapse, neither quantity is observable. In this first regime, memory and computation are free in the technical sense that they are not charged or exported as resources.

This choice matches the first case in the modeling taxonomy of [3, Sec. 3.5]. If converter implementation complexity is irrelevant, one quantifies over converter systems without regard to that complexity. The same section then considers different models in which memory or computation is relevant and must be represented explicitly as part of the resource.

10.2 The Ristenpart-Shacham-Shrimpton lesson

Ristenpart, Shacham, and Shrimpton showed that unrestricted composition can fail for a form of indifferentiability when state retained across uses carries information between contexts [9]. Maurer and Renner explain the associated modeling issue as follows: an ordinary Turing machine comes with arbitrary tape memory, so it is the wrong converter model when memory availability is the resource whose limitations are intended to support the claim [3, Sec. 3.5].

There are two distinct conclusions.

First, the example is not a reason to remove persistent tape from the initial unbounded model. In that model memory has deliberately been erased, and a stateful converter is admissible. One must simply refrain from reading the result as a bounded-memory guarantee.

Second, when a theorem depends on erasure, isolation, reset behavior, or a capacity bound, memory must cross the abstraction boundary. Hiding it inside a Turing tape and then reasoning as if it were unavailable is a modeling error. The counterexample diagnoses a mismatch between the desired resource accounting and the chosen lower model.

10.3 The refinement path

The top-down hierarchy suggests refinements rather than a replacement of the current layer.

Relevant cost	Possible lower resource model
Persistent memory	Converters become stateless routers that read and write an explicit memory resource with capacity, leakage, reset, or erasure interfaces.
Computation	A processor resource accepts a program and data, performs at most a stated number of steps, and returns a result; converters only route and select programs.
Time	Clock ticks and deadlines become visible events, or messages carry positions in a time order.
Randomness	Machines query an explicit seed, beacon, or random-bit resource rather than receiving a private free tape.
Physical leakage	The implementation resource exports specified leakage observations; no claim is inferred from the purely functional DDS.

Jost gives concrete examples of memory and randomness resources in a constructive treatment [8, Sec. 6.2.3]. The abstract reduction viewpoint also makes clear why an unconditional solver bound need not mention implementation complexity [4, Secs. 4.4.3-4.4.5].

A refined cost-aware model should come with a forgetful homomorphism that erases the added observations and returns the unbounded operational model. Then every high-level theorem remains available after forgetting costs, while new lower-level theorems can mention them. The next sections construct the machine-cost and efficient-family refinements. A later section then fixes and proves one private sequential processor/store/coin API. Its exact interface is a selected lower model rather than a canonical choice for every application.

11 Exact cost traces

11.1 Refining the run, not the random system

The macro semantics was designed to forget implementation details. It is therefore too late to attach a cost after reaching a DDS. Two extensionally equal history functions may be implemented by programs with arbitrarily different running times, and an abstract history function need not have a program at all. Cost belongs to the small-step realization before macro-collapse.

We retain the operational core of Section 4 and label its primitive events. The resulting trace has both a behavioral projection and a cost projection:



Figure 3: Behavior and cost are two projections of the same operational run.

No polynomial appears yet. A ledger records what a particular execution used; a grade introduced later bounds those records.

11.2 Bit-costful machines and routers

The machine convention needs one refinement. Every wire value is encoded by an effective self-delimiting bit string, including its port and administrative tags. At the beginning of an activation the output buffer is blank. A transition changes only a fixed number of cells, and there is no pointer operation that aliases an input or persistent work buffer as the next output. Re-emitting a stored ℓ -bit value therefore takes $\Omega(\ell)$ transitions.

Routing is not free metanotation. A canonical router validates the tag, moves or copies the finite code according to a fixed tape convention, clears the old buffer, and activates the receiver. Router transitions and transmitted bits are charged. A hardware model with direct buffer ownership transfer is possible, but it is a different named cost model.

An external input is itself an output of the surrounding context. Thus a closed experiment pays for producing and routing all messages. These conventions prevent a repeater from creating or forwarding an exponentially long value in one step.

11.3 The exact ledger

Let ρ be a finite run prefix. Treat each canonical router or other administrative mechanism as a named infrastructure occurrence. For every machine or infrastructure occurrence v , edge e , and named ideal-resource port j , define

$$\text{Cost}(\rho) = \left((\text{step}_v)_v, (\text{act}_v)_v, (\text{rand}_v)_v, (\text{peak}_v)_v, \text{gpeak}, (\text{bits}_e)_e, (\text{calls}_j)_j, (\text{qbits}_j)_j, (\text{rbits}_j)_j \right). \quad (40)$$

Definition Primitive cost coordinates. The coordinates in Equation 40 have the following meanings.

- step_v counts Turing transitions for a machine occurrence or canonical-routing transitions for the corresponding infrastructure occurrence v .

- act_v counts deliveries that activate v .
- rand_v counts bits read from its named random tape.
- peak_v is the largest native logical live footprint reached by v . The selected convention counts occupied tape and register cells and one logical cell for each fixed finite control/state record and tape-head record; the mode and pending finite port tag may be packed into the control record. A numeric head coordinate is one logical record here; the bit length of its later physical serialization is a separate lower cost.
- gpeak is the largest **simultaneous** total $\sum_v \text{live}_v$ over all small-step configurations in the prefix, including occupied in-flight canonical-routing buffers.
- bits_e is the total encoded traffic over edge e .
- calls_j counts admitted calls to named abstract resource j .
- qbits_j and rbits_j separate its query and response traffic.

The primitive-event boundary is fixed as follows. From the validated self-delimiting header, a canonical router first reserves the complete message's copy-work, edge-traffic, and buffer envelope. Rejection terminates with the router owner before any routing coordinate or destination buffer is committed. On admission, the router performs the ordinary bit-costful copy in small steps under that capability; those exact transitions, transmitted bits, and intermediate buffers are committed and cannot subsequently exhaust. Receiver delivery is then one receiver-owned activation event: it jointly increments act_v , installs the validated input code, and updates peak_v to the resulting live footprint. A rejected receiver charge leaves the receiver suspended and does not install the input; already completed routing cost remains in the prefix ledger. After every intermediate or committed configuration, the external accountant updates gpeak from the actual simultaneous live state. Tentative state that has not been installed is excluded from the native coordinate; an explicit lower refinement may report its physical scratch separately.

Likewise, one machine transition jointly updates step_v , its prospective peak, and rand_v iff it inspects the current random cell. For fixed tapes the meter may determine this prospective charge without exposing it to the program. If the joint charge is rejected, the transition does not occur and the random head does not advance. These conventions fix the ordering needed for exact failure and refinement claims; another delivery or tape-access convention is a different cost model.

Peak space is not additive in time. The exact ledger is consequently a structured record, not one scalar monoid element. A global report may derive

$$\text{Work} = \sum_v \text{step}_v, \quad \text{Acts} = \sum_v \text{act}_v, \quad \text{Coins} = \sum_v \text{rand}_v, \quad (41)$$

$$\text{Traffic} = \sum_e \text{bits}_e, \quad \text{Space} = \text{gpeak} = \max_t \sum_v \text{live}_{v(t)}. \quad (42)$$

Other monotone aggregations may represent per-party cost, energy, latency, or hardware prices. The aggregation is part of the chosen cost model and must be named in a concrete theorem.

Proposition Exact-ledger uniqueness. For a well-typed network, fixed random tapes, fixed selected ideal-resource seed sequences, and a finite run prefix, the ledger Equation 40 is unique.

Proof. The single-token rules determine a unique labeled small-step trace. Every transition, activation, random-tape read, emitted code, and resource call has a fixed coordinate update. Folding those labels over the unique prefix gives one record. $\square$

Proposition Domination by bit-level work. In the machine-only model, newly visited cells, random bits read, emitted bits, and activations are each bounded by a fixed linear function of the number of charged transitions. Ideal-resource response bits are excluded and are charged separately.

Proof. A transition changes or visits only a fixed number of cells, reads only a fixed number of random bits, and writes only a fixed number of output bits. Every activation and routing action includes at least one charged transition. Summing the corresponding local constants proves the bounds. $\square$

The separate coordinates remain useful despite this domination. Concrete reductions count primitive queries, communication can be priced differently from local work, and a later memory resource makes capacity observable.

11.4 Lifetime accounting

Ledger state persists across macrosteps just as work tapes and random-tape positions persist. A blocked finite run has a finite terminal ledger. A divergent run has a directed family of finite-prefix ledgers, with an unbounded positive work or routing coordinate. It is not assigned a completed infinite computation.

For the persistent-mask example, fix the following elementary implementation. Storing the outer bit and emitting the one-bit key request costs four work steps; each one-bit route costs five; the first activation of K samples, stores, copies, and emits in five steps, while a later activation copies the stored bit and emits in three; finally C reads its two bits, computes xor, and emits in five. Activations are counted separately from work. The first outer query therefore has aggregate ledger

$$\ell_1 = (24, 3, 1, 2, 4), \quad (43)$$

in the order (work, activations, random bits, internal traffic, peak live cells). Every later query adds $(22, 3, 0, 2, 0)$, where zero in the last coordinate means that the lifetime maximum remains four. After $q \geq 1$ queries,

$$\ell_q = (22q + 2, 3q, 1, 2q, 4). \quad (44)$$

This is one certified implementation cost, not a machine-independent optimum. It is the fixed unparameterized instance. When the same two occurrences are placed in the primary uniform-family convention of Section 14, their two logical read-only 1^κ tracks and heads coexist with these four protocol cells, so the peak coordinate becomes $2\kappa + 6$; work, activations, randomness, and traffic are unchanged. Together with Equation 5 it also illustrates why lifetime state cannot be replaced by a per-invocation clock: fresh masking has the same one-query distribution, while the equality test for all recovered masks distinguishes the two resources with exact advantage $1 - 2^{1-q}$.

Proposition Cost invariance under network isomorphism. A typed graph isomorphism fixing the boundary induces a bijection of ledger coordinates and preserves every exact cost after that relabeling.

Proof. The isomorphism maps components, edges, tapes, and transition codes bijectively and preserves the active-token relation. Induction on a run prefix gives the same labeled event in the corresponding coordinate. $\square$

Before exposing cost to a context, fix a measurable **public report** map Rep_N that is invariant under every internal alpha-isomorphism:

$$\text{Rep}_{\varphi_N}(\varphi_*\ell) = \text{Rep}_{N(\ell)}. \quad (45)$$

It may retain coordinates carrying declared public party, interface, oracle, or ownership labels, but cannot reveal a freshly chosen internal occurrence name. At the pointwise cost layer, a maximal option is the alpha-orbit of the finite graph/ledger pair. In the computational class, the selected projection must instead have the fixed effective, output-length, and charged-evaluation certificate stated in Section 16; no efficient graph canonizer is silently supplied.

12 Metered execution

12.1 Why a local PPT slogan is insufficient

Interactive polynomial time has several non-equivalent readings. Three naive ones fail before any cryptographic security definition is considered.

Counterexample Polynomial per activation. Machine A maps an input string x to xx ; machine B repeats its input. Both take time linear in the current input. Connect their outputs in a cycle. After t visits to A ,

the circulating message has length 2^t . Every activation is locally polynomial, while cumulative work is exponential.

Counterexample Polynomial input shape. A forwarder and a repeater are each linear in traffic received from outside the component. Connected on one branch, one external input can circulate forever. Thus a polynomial input-output shape is not closed under arbitrary connection. This is the standard forwarder/repeater obstruction in reactive runtime [10], [11].

Counterexample One lifetime clock. Give an ideal database a lifetime bound κ^3 . An environment making κ^4 harmless queries exhausts it, although every query has an efficient answer. Selecting a larger fixed exponent only moves the problem to another polynomial environment.

The first two failures concern global flow. The third concerns quantifier order. A reactive service must scale with the workload of each fixed efficient context, but cycles must not manufacture additional work.

12.2 Ambient workload and component grades

We introduce a public administrative workload $b \in \mathbb{N}$. It is not a wire message and programs cannot inspect it. A closed efficient context will later choose $b = p(\kappa + |a|)$ for one fixed polynomial p and public input a .

Definition Polynomial component grade. A grade for component occurrence v is a finite tuple of monotone multivariate polynomials

$$F_v : (\kappa, b) \mapsto B_v \quad (46)$$

whose coordinates bound transitions, live cells, random bits, traffic, activations, and named ideal-resource calls. Exponents and coefficients are fixed in the finite grade description.

Grades label occurrences in the normalized final graph. They are transported by alpha-renaming and do not depend on the parenthesization used to build the graph. A component's program may attempt more work; its meter defines the bounded behavior.

The ambient workload solves the database quantifier problem. A higher-degree context selects a larger polynomial b , and every fixed component transformer scales with it. For each fixed context the resulting bound is still a polynomial in κ .

12.3 Non-observable meters

Before a primitive event occurs, an external meter computes the next ledger. If a component or infrastructure quota would be crossed, execution terminates with a lower status. Programs cannot read their quota, remaining credit, or the exhaustion event.

Primitive charge events have one declared public owner. All local coordinates updated by one machine step therefore attribute a simultaneous multi-coordinate failure to the same owner. A cost model that also imposes cross-owner global caps must give a fixed alpha-invariant priority among the rejecting meters; that priority is part of the model and makes the terminal label unique.

Definition Metered macro result. A metered macro execution has one of the results

$$\text{Success}(c', y, \ell), \quad \text{Block}(v, \ell), \quad \text{Exhaust}(v, \ell), \quad (47)$$

where ℓ is the lifetime ledger and v is an alpha-invariant public ownership class for the blocking or exhausted occurrence. Exhaust is not a response in any ordinary wire alphabet.

In a closed test, the terminal outcome also retains the complete finite visible transcript τ and a designated finite observation o of the test's own state. Ownership classes distinguish tested code, context code, and declared shared infrastructure; they do not reveal fresh internal names. Plugging and converter absorption preserve them.

Every machine and router transition has positive work cost. An ideal-resource call is atomic at this layer but consumes a call tariff defined in Section 15. Strict nonresponse from that resource gives Block.

Theorem Finite-meter termination. With a finite positive work quota, every machine-only metered macro execution reaches Success, Block, or Exhaust after finitely many small steps.

Proof. If success or blocking does not occur within the work quota, the next positive-cost transition would cross it and the external meter returns Exhaust. $\square$

12.4 Monotonicity and stabilization

Theorem Budget monotonicity. Fix all random tapes, selected ideal-resource seed sequences, and initial specification states. If a run succeeds under quota vector B and $B \leq B'$ coordinatewise, then it follows the same trace and succeeds with the same output and successor state under B' .

Proof. Induct over the finite successful trace. Every prospective charge or pre-reservation vector checked along that trace fits below B , hence below B' . The meter therefore makes the same admission decision. Since the program cannot inspect the quota, it selects the same next transition at every prefix; fixed oracle seeds then select the same admitted outcomes. $\square$

Definition High-water funding requirement. For a finite fixed-sample unmetered run ρ , let $\text{Need}(\rho)$ be the coordinatewise maximum of every absolute vector that the corresponding metered execution would test along the trace. This includes prospective primitive ledgers, each reserve-evaluator envelope and each current ledger plus an oracle's combined semantic/post-sample administrative reservation, and any other declared atomic transaction reservation. It also includes the terminal exact ledger.

Theorem Finite-success stabilization. If an unmetered fixed-sample execution succeeds along ρ , every budget $B \geq \text{Need}(\rho)$ gives the same metered success. Conversely, every metered success is the same success of the unmetered network. In the machine-only model, or whenever every reservation equals its realized prospective charge, $\text{Need}(\rho) = \text{Cost}(\rho)$. In general only $\text{Cost}(\rho) \leq \text{Need}(\rho)$ is guaranteed.

Proof. In the first direction every check vector used on ρ fits below B . Induction therefore reproduces the same transition, oracle outcome, and released reservation at each step. In the second, remove meter checks from the finite successful witness. Machine charges are cumulative or peak updates, so their prospective check vectors are bounded by the terminal ledger. A public oracle envelope or a transactional prepare may be strictly larger than the exact charge it eventually commits, which proves the final qualification. $\square$

Thus meters refine rather than approximate sufficiently funded finite computation. The high-water distinction is essential for a pre-sample oracle: an exact response charge can be smaller than the public envelope that had to fit before its seed was read. An unmetered micro-divergence or infinite hidden token transfer exhausts every finite positive work budget and erases to nonresponse.

12.5 Fixed-graph polynomial boundedness

Theorem Polynomial bound for a metered graph. Let N be a fixed finite normalized graph. Suppose every component grade and the canonical infrastructure grade are polynomial in (κ, b) , and every specification occurrence has the polynomial reservation and evaluator envelopes called efficient accessibility in Section 15. If a fixed context chooses $b = p(\kappa + |a|)$ for a polynomial p , every closed metered run has a polynomially bounded global ledger.

Proof. Each coordinate is bounded by a finite sum or maximum of the component and infrastructure quotas; admitted specification charges lie below their evaluated reservations. A finite sum, maximum, or composition of fixed monotone polynomials has a polynomial upper bound. Substituting the fixed polynomial p preserves polynomiality. Wiring can change which quotas are spent but cannot change their values. $\square$

The theorem includes cyclic feedback. A bad loop may consume its quotas and exhaust, but cannot mint further work. It proves boundedness, not availability.

12.6 Boundedness is not responsiveness

A machine that spends its entire quota without answering satisfies the polynomial-bound theorem. To implement a responsive specification, it needs a second judgment: on the advertised workload, a visible answer occurs before any quota is exhausted.

Sufficient proof disciplines include:

1. an acyclic hidden-call graph with either response-independent elementary bounds or a solved response-adaptive size invariant;
2. a well-founded rank that decreases on each hidden transfer;
3. consumable credit that decreases on every cycle;
4. a polynomial fixed-point certificate for internal traffic in the particular composition.

The metered implementation algebra is closed under every well-typed finite connection. The adequately responsive subclass is closed only under such additional hypotheses. This distinction is the efficient analogue of partial closure in Section 9.

12.7 Adequacy and its quantifiers

No-exhaustion cannot quantify over an unlabeled global stop in every metered context. A context can deliberately burn its own quota and thereby make every resource fail. We therefore fix a class $\mathcal{C}$ of **completion contexts**. Each member has a workload policy, an interface envelope for responses it is prepared to receive, a safety certificate for its own occurrences under that envelope, and a progress certificate saying that it does not deliberately block or stop silently and reaches its designated decision when the tested side meets the envelope. The terminal owner label separates Exh_X (tested-side), Exh_D (context-side), and Exh_{sh} (shared infrastructure).

Let $\chi_{D(\kappa)}$ bound the union of the completion context's certified safety failure, its progress failure when the tested side respects the response envelope, and declared shared-infrastructure failure. It may be taken as the sum of those three bounds. A constructor theorem showing that the tested graph answers must add χ_D before concluding that the **closed** experiment reaches Success; in the strong completion-context variant, $\chi_D = 0$.

Definition Four implementation judgments. Fix an input mode and completion-context class $\mathcal{C}$.

- **Metered boundedness** means that every closed run has a polynomial ledger and ends in success, block, or owner-labeled exhaustion.
- **Overwhelming no-exhaustion** means that for every fixed $D \in \mathcal{C}$ there is a negligible ν_D such that

$$\sup_a P[\text{Exh}_{X(D, X, \kappa, a)}] \leq \nu_{D(\kappa)}. \quad (48)$$

Strong and almost-sure variants replace this probability bound by an empty event or probability zero.

- **Productivity** on a declared responsive domain means that every fixed completion context reaches success except with negligible probability.
- **Behavioral realization** compares the erased transcript law with the target specification in the selected observer metric.

Inputs are not hidden inside the probability. In pure uniform mode, one fixed generator produces a . In bounded auxiliary-input mode the supremum ranges over every $|a| \leq q_{D(\kappa)}$, which has nonuniform advice strength. In an explicitly nonuniform mode the advice sequence is itself part of the quantified object.

Proposition Adequacy implications and attribution. Strong no-exhaustion implies almost-sure no-exhaustion, which implies overwhelming no-exhaustion. Productivity implies overwhelming no-exhaustion on the same completion class. Moreover,

$$P[\text{Exh}_{\text{all}}] \leq P[\text{Exh}_X] + \nu_D^{\text{ctx}} + \nu_D^{\text{sh}}. \quad (49)$$

Proof. Empty events have probability zero, zero is negligible, and tested-side exhaustion is one way not to succeed. The global event is contained in the union of the three owner classes. $\square$

These implications do not reverse. Spin is metered-bounded and exhausts with probability one. A one-step silent blocker never exhausts but is not productive. A program can answer efficiently with the wrong bit, so productivity does not imply realization. Conversely, a program that always exhausts realizes a strict-nonresponse target after erasure but is not an adequate implementation. Expected polynomial work is also insufficient: a fixed program reads $m = \lceil \log_2 \kappa \rceil$ fair bits and performs κ^3 further steps iff they are all zero. The long branch has probability $2^{-m} \in [\frac{1}{2\kappa}, \frac{1}{\kappa}]$, so expected work is at most $\kappa^2 + O(\log \kappa)$, but a κ^2 meter exhausts with nonnegligible probability at least $\frac{1}{2\kappa}$.

For fair-bit machines, a finite exhaustion has a finite positive-probability cylinder. Almost-sure and strong no-exhaustion therefore coincide on reachable machine samples. The same argument with specifications requires a **positive-branch presentation**: the initial-state draw and every oracle call, at every reachable complete history, have a countable branch label such that (i) the label determines all successor information relevant to subsequent finite operational behavior and (ii) every label in the selected sampler's range has strictly positive conditional mass. Responses and public coordinates alone do not suffice when hidden successors with the same public view have different futures. Under this premise a finite exhaustion witness fixes finitely many fair bits and branch labels and hence has positive probability. A continuous hidden null branch, including one in the initial state, defeats the implication even if the response alphabet is finite. Separately, almost-sure **unmetered termination** does not imply almost-sure no-exhaustion at any finite grade: the program that flips until the first one terminates almost surely, yet exceeds every fixed finite coin/work budget with positive probability.

Proposition Metered/unmetered coupling. Couple a metered and unmetered closed execution with the same machine tapes oracle seeds, and initial specification states. Their configurations, ledger prefixes, and visible events agree until the first failed meter check. Hence, on finite transcript outcomes,

$$d_{TV}(\text{erase}(\text{Run}_{\text{metered}}), \text{Run}_{\text{unmetered}}) \leq P[\text{Exh}_{\text{all}}]. \quad (50)$$

This bound keeps semantic error, exhaustion, and nonproductivity distinct in a concrete construction statement.

12.8 Two constructor certificates

The generic algebra cannot make the responsive subclass closed, but useful compositions admit local certificates.

Theorem Acyclic hidden-call adequacy. Let the hidden-call graph be a fixed DAG. For every occurrence v , suppose fixed polynomials bound local work t_v , response length r_v , each child query length m_{vw} , child-call count q_{vw} , and conditional local failure δ_v , uniformly over every certified adaptive history. Suppose u_v bounds one active or suspended transient frame and $p_{v(\kappa, b, N, n, (R_w))}$ bounds all state retained by occurrence v after at most N certified activations. The latter includes parameter tracks, heads, and accumulated tables; it is not inferred from a one-activation peak. Suppose the elementary query-length and call-count bounds do not depend on returned child values. A local failure is the first violation of the advertised progress, size, or ledger contract; before that failure, every activation either returns visibly or issues one of its certified child calls.

Initialize N_v, L_v for roots from the advertised external workload. For each nonroot w , compute topologically

$$N_w = \sum_{v \rightarrow w} N_v q_{vw}(\kappa, b, L_v), \quad L_w = \max_{v \rightarrow w} m_{vw}(\kappa, b, L_v), \quad (51)$$

then compute reverse-topologically

$$R_v = r_{v(\kappa, b, L_v, (R_w)_{v \rightarrow w})}. \quad (52)$$

A meter dominating

$$\text{Work}_{\text{DAG}} = \sum_v N_v t_{v(\kappa, b, L_v, (R_w)_{v \rightarrow w})} \quad (53)$$

and the analogous cumulative ledger coordinates, together with

$$\text{Persist}_{\text{DAG}} = \sum_v p_{v(\kappa, b, N_v, L_v, (R_w))}, \quad (54)$$

$$\text{Stack}_v = u_{v(\kappa, b, L_v, (R_w))} + \max_{v \rightarrow w} \text{Stack}_w + \text{RouteStack}_v, \quad (55)$$

$$\text{Space}_{\text{DAG}} = \text{Persist}_{\text{DAG}} + \max_{v \text{ root}} \text{Stack}_v + \text{RouteBase}_{\text{DAG}}, \quad (56)$$

makes the **tested graph** strongly answer without tested-side exhaustion for worst-case local certificates. In a closed completion experiment its failure is at most χ_D . With probabilistic certificates, tested-side failure is at most $\sum_v N_v \delta_{v(\kappa, b, L_v)}$ and closed failure is at most that sum plus χ_D , relative to environments obeying the advertised root workload and interface envelope.

Proof. The two topological passes bound every activation, query, and response. Induction from the sinks proves the local work envelope. The conditional failure premise permits an adaptive stopped union bound: order the deterministic activation slots, expose only the event that a slot is the first local failure, and condition on the preceding certified history. Before the first failure the displayed N_v bounds the number of exposed slots. Independence is not used, and behavior after the first failure is irrelevant because the meter still bounds it. Response-adaptive query growth instead requires an explicitly solved size recurrence and is not covered by this elementary rule. $\square$

Theorem Response-adaptive DAG adequacy. Retain the fixed DAG, but let the next-query and total-call polynomials $M_{vw}(\kappa, b, n, z)$ and $Q_{vw}(\kappa, b, n, z)$ depend on the cumulative child-response length z already seen in the current activation. Let $\Delta_{v(\kappa, b, n, z)}$ bound the conditional probability that the local progress, size, or ledger certificate first fails, uniformly over every certified history with cumulative response length at most z . Let $u_{v(\kappa, b, n, z)}$ bound active/suspended transient state and $p_{v(\kappa, b, N, n, z)}$ bound lifetime persistent state after at most N certified activations. Process children before parents. If each v supplies a fixed polynomial $Z_{v(\kappa, b, n)}$ satisfying

$$\sum_{v \rightarrow w} Q_{vw}(\kappa, b, n, Z_v) R_{w(\kappa, b, M_{vw}(\kappa, b, n, Z_v))} \leq Z_v, \quad (57)$$

then $z \leq Z_v$ is an invariant. In particular one may define

$$R_v = r_{v(\kappa, b, n, Z_v)}, \quad (58)$$

$$T_v = t_{v(\kappa, b, n, Z_v)} + \sum_{v \rightarrow w} Q_{vw}(\dots, Z_v) T_{w(\kappa, b, M_{vw}(\dots, Z_v))} + \text{Route}_v, \quad (59)$$

$$\text{Stack}_{v(\kappa, b, n)} = u_{v(\kappa, b, n, Z_v)} + \max_{v \rightarrow w} \text{Stack}_{w(\kappa, b, M_{vw}(\dots, Z_v))} + \text{AdminStack}_v. \quad (60)$$

Cumulative coordinates use the same sum pattern, with the empty maximum equal to zero. Route_v and AdminStack_v are the fixed routing and suspended-frame certificates. A forward pass with Q_{vw} evaluated at Z_v gives activation bounds N_v and input bounds L_v . Put

$$\text{Persist}_{\text{RA}} = \sum_v p_{v(\kappa, b, N_v, L_v, Z_{v(\kappa, b, L_v)})}, \quad (61)$$

$$\text{Space}_{\text{RA}} = \text{Persist}_{\text{RA}} + \max_{v \text{ root}} \text{Stack}_{v(\kappa, b, L_v)} + \text{RouteBase}_{\text{RA}}. \quad (62)$$

All persistent occurrence states coexist; only transient frames take a root-to-leaf maximum. These are polynomial aggregate grades. For worst-case certificates the tested graph strongly answers without tested-side exhaustion. In the probabilistic case the probability that any local certificate fails is at most

$$\text{Fail}_{\text{RA}} = \sum_v N_v \Delta_{v(\kappa, b, L_v, Z_{v(\kappa, b, L_v)})}. \quad (63)$$

This is the tested-side bound. Closed completion failure is at most $\text{Fail}_{\text{RA}} + \chi_D$; hence closed productivity is overwhelming when both terms are negligible, relative to the same root workload and interface envelope. Strong closed productivity additionally requires $\chi_D = 0$.

Proof. Suppose one activation first crosses Z_v . Before that response, monotonicity bounds every query by $M_{vw}(\dots, Z_v)$, every child response by the reverse-inductively known R_w , and the total number of calls by $Q_{vw}(\dots, Z_v)$. Their total encoded length is at most the left side of Equation 57, contradicting the crossing. Hence the invariant holds. Reverse induction over the fixed DAG bounds each activation; the ordinary forward pass bounds activation multiplicity. Cumulative coordinates sum child envelopes. The forward multiplicities fund every persistent certificate; summing those coexisting states and adding the maximum active transient stack proves the global space display. All are fixed polynomial compositions. For the probabilistic claim, stop at the first failed local certificate. Every earlier activation is certified, so the deterministic forward count still bounds the possible first-failure slots. Conditioning on each preceding certified history and summing their bounds proves the displayed failure estimate without independence. $\square$

The polynomial Z_v and proof of Equation 57 are certificate data, not automatic consequences of local polynomial time. A parent that makes κ calls to an echo child and squares the preceding response length before the next call has an acyclic call graph but reaches $2^{2^{\kappa-1}}$ bits. Its natural recurrence cannot have a nonzero polynomial post-fixed point. Thus the stronger rule admits adaptive calls without admitting polynomial iteration by slogan.

For a positive instance, suppose a child reply has public length at most $\lambda(\kappa, b)$ independently of its query, a parent makes at most $q(\kappa, b, n)$ calls, and each next query may depend on all previous replies. Then $Z_v = q\lambda$ satisfies Equation 57, while the adaptive query bound is safely evaluated at $z = q\lambda$. The rule therefore permits genuine adaptive content and size dependence when a polynomial cumulative envelope exists.

Theorem Affine-credit feedback adequacy. Suppose a fixed cyclic graph begins a macro execution with polynomial credit $C_0(\kappa, b)$. Uniformly over every reachable history and remaining credit, every activation terminates within $T_{\max}$ without blocking or exhausting and either emits visibly or makes exactly one hidden transfer; every hidden transfer strictly decreases credit, no hidden step creates credit, and credit zero forces the next activation to emit visibly. If $L_{\max}$ bounds every hidden and visible message and $S_{\max}$ bounds an active occurrence's transient state, require also a polynomial $P_{v(\kappa, b, C_0)}$ bounding every state cell retained by occurrence v between activations over the certified macro execution. Then before the visible response there are at most C_0 hidden transfers and $C_0 + 1$ activations, with

$$\text{Work} \leq (C_0 + 1)T_{\max} + \text{Routing}(C_0 + 1, L_{\max}). \quad (64)$$

Traffic is at most $(C_0 + 1)L_{\max}$ up to the fixed encoding and routing constants; cumulative randomness and oracle-call coordinates satisfy the analogous activation sum. Peak space is bounded by $\sum_v P_v + L_{\max} + S_{\max}$ plus the fixed routing stack. Thus inactive persistent states are summed rather than hidden behind the one active transient maximum. All displayed quantities are polynomial.

In the probabilistic variant, let δ_v bound conditional local certificate failure at occurrence v , uniformly over every certified adaptive history. The probability of any failure is at most $(C_0 + 1) \max_v \delta_v$, or the sharper sum over certified occurrence activation counts. These are tested-graph bounds. Closed completion failure additionally includes χ_D ; hence polynomial credit and negligible local and completion-context bounds give overwhelming productivity. Strong closed productivity requires $\chi_D = 0$.

Proof. Credit is a natural-valued variant decreasing at every hidden traversal. Sum the local and routing envelopes; the zero-credit rule excludes blocking at the final activation. For probabilistic certificates, stop at the first failure. Every preceding activation decreased credit or answered, so there are at most $C_0 + 1$ candidate first-failure slots. Condition on each certified prefix and apply the union bound; no independence is required. $\square$

The credit can be a ghost variant proved from ordinary machine state; then it has no run-time representation. If an implementation instead carries and updates a credit counter, its encoding, head position, update work, and message field are ordinary charged state and traffic. In neither reading may the program inspect an otherwise hidden meter quota or learn exhaustion.

Example A complete credit-guarded cycle. Put $x = \kappa + b$. The external request supplies a unary count-down z with $|z| \leq x$. A fixed two-node cycle deletes one symbol on every hidden transfer and emits visibly when the string is empty. Thus $|z|$ is a ghost credit derived from ordinary message state; neither node reads b or a meter. The static certificate may safely use $C_0 = x$. Suppose each activation uses at most $7x + 11$ work and each hidden or visible message has length at most $x + 3$. With router cost $\text{Route}(L) = 2L + 3$, the complete work certificate is

$$(x + 1)(7x + 11) + (x + 1)(2(x + 3) + 3) = 9x^2 + 29x + 20. \quad (65)$$

Traffic is at most $(x + 1)(x + 3)$ and there are at most $x + 1$ activations. Let each of the two nodes retain at most $\kappa + 5$ logical cells, let the one active transient frame use at most $x + 4$, and reserve $x + 3$ cells for the one live message and eight for routing control. Then

$$\text{Space} \leq 2(\kappa + 5) + (x + 4) + (x + 3) + 8 = 4\kappa + 2b + 25. \quad (66)$$

Random-bit and named-oracle-call coordinates are zero. These meters never fire on the certified domain, and the credit rule proves a visible response rather than merely truncating the cycle. A represented credit counter would add its ordinary storage and update work.

12.9 Two observation levels

A lower supervisor can retain the terminal status and ledger. An ordinary random-system context cannot. Define

$$U(\text{Success}(\tau, o, \ell)) = \text{Visible}(\tau, o), \quad (67)$$

$$U(\text{Block}(v, \tau, o, \ell)) = U(\text{Exhaust}(v, \tau, o, \ell)) = \text{NoResponse}(\tau, o). \quad (68)$$

NoResponse retains the maximal visible prefix and the closing test's own observation, but is not a response symbol that can be fed back to a program. The costed carrier can express report-, owner-, or exhaustion-sensitive tests; behavioral tests factor through U .

The identity at the costed layer is a typed bijection between administrative boundary faces, not a relay and not an arbitrary equality of ports. Normalization composes alias bijections, so identity syntax creates no component, fanout, or second routing charge. Physical transport is charged once to a remaining ordinary edge or explicit communication resource.

12.10 The finite costed operational algebra

At fixed (κ, b) , let $\text{Graph}(B)$ be the finite normalized open graphs with boundary B , evaluated grades, public ownership classes, and admitted specification nodes. A cost-aware closing context applies a total measurable bit decision to $(\tau, o, \text{status}, \text{owner}, \text{Rep}(\ell))$, for the fixed isomorphism-invariant report above. Define $G \stackrel{\text{cost}}{=} H$ when every such context has the same decision law; define $\stackrel{\text{beh}}{=}$ using only decisions that factor through Equation 68.

Theorem Finite operational system algebra. The quotients

$$\mathcal{R}_{\kappa, b}^{\text{cost}}(B) = \frac{\text{Graph}(B)}{\stackrel{\text{cost}}{=}}, \quad \mathcal{R}_{\kappa, b}^{\text{beh}}(B) = \frac{\text{Graph}(B)}{\stackrel{\text{beh}}{=}} \quad (69)$$

admit typed renaming, symmetric finite parallel composition, every legal finite connection, and converter action. Connection order and parenthesization do not change the quotient element, including when the final matching contains cycles.

Proof. Alias normalization terminates because each rewrite removes one alias and is confluent because typed bijection composition is associative. Every constructor is a one-hole context after its other arguments are fixed, so contextual equivalence is a congruence. Different construction orders have the same alpha-disjoint node union and final endpoint matching; canonical edge names depend only on endpoint pairs. Finite meters give every cyclic closed graph a terminal kernel, possibly by owner-labeled exhaustion. $\square$

Theorem Erasure homomorphism. Cost equivalence implies behavioral equivalence, and

$$U_B : \mathcal{R}_{\kappa,b}^{\text{cost}}(B) \rightarrow \mathcal{R}_{\kappa,b}^{\text{beh}}(B) \quad (70)$$

preserves renaming, parallel composition, connection, converter action, and identity.

Proof. Every behavioral test is a cost-aware test that ignores the report and factors through Equation 68. Erasure changes the terminal observation, not the normalized graph on which each constructor acts. $\square$

This is the complete finite bounded algebra. It deliberately permits an underfunded physical matching. Such a matching may exhaust and therefore cannot be identified with cost-free strict DDS feedback. The next subsection isolates the closed subalgebra on which the further random-system map is valid; a fallible link remains expressible as an explicit resource node.

13 Route-safe bridge to partial random systems

Let the fixed canonical router use at most $a_0 + a_1 L$ work, L traffic bits, and $a_{S(L+1)}$ live cells for an encoded event of length L . The blank-output bit-machine convention gives fixed constants c_0, c_1 such that machine event count and emitted bits are bounded by $c_0 W$ and $c_1 W$ when aggregate nonrouter machine work is W . Let C bound admitted specification events and let R bound their aggregate encoded query, response, and block-record traffic. Define

$$\text{RouteWork} = a_0(c_0 W + C) + a_1(c_1 W + R), \quad (71)$$

$$\text{RouteTraffic} = c_1 W + R, \quad \text{RouteSpace} = a_{S(1+c_1 W+R)}. \quad (72)$$

When component grades and access envelopes are fixed polynomials in (κ, b) , so is Equation 72; no parameter-dependent exponent or router code is introduced.

Definition Route-safe graph. A normalized graph is **route-safe** when each canonical structural matching receives at least the whole-graph envelope Equation 72, evaluated from the final aggregate component and specification grades. Equivalently, one aggregate infrastructure meter may enforce that envelope while the ledger retains per-edge charges. A bandwidth-limited channel, fallible link, or memory-limited buffer is instead an explicit node with an ordinary interface and meter.

The redundant per-edge assignment avoids assuming one privileged global router: total canonical-routing use is below Equation 72, hence every edge's local use is below it. Only matched edges are charged. An open-boundary input is delivered directly; after a context is attached, its machine must produce the query before the new matching routes it.

Proposition Routing transparency. A canonical router in a route-safe graph cannot be the first exhausted occurrence. Route-safe graphs are closed under renaming, finite parallel composition, connection, and converter action by recomputing Equation 72 on the final normalized graph.

Proof. Every routed event is emitted by a charged machine transition or by one admitted specification call. Sum the event-count and bit bounds over all canonical edges; each local router is bounded by the same aggregate. The one-token discipline leaves at most one active routing buffer, giving the space bound. Bare aliases have been removed; hence an infinite hidden chain consumes positive machine work or a positive specification-call coordinate, and a nonrouter meter fires first. Recomputing the external, program-hidden grade changes no transition or ledger event, and every construction order has the same final aggregate. $\square$

For countable X, Y , embed a partial DDS in

$$(Y + \{\perp\})^{X^+}. \quad (73)$$

Prefix closure is a Borel condition: its failure is the countable union of cylinders in which a value is defined while one prefix is undefined. Thus $\text{DDS}_{\partial(X,Y)}$ is standard Borel and its Borel sigma-algebra is generated by the evaluation events of Equation 16.

To observe random domains without adding an error symbol, use **maximal strict transcripts**. An interaction is either an infinite query-answer stream, a finite environment stop, or

$$\text{SysStop}((x_1, y_1), \dots, (x_n, y_n); x_{n+1}), \quad (74)$$

which retains the query issued without a response. These outcomes form a standard-Borel disjoint union. Every deterministic environment induces a measurable maximal-run map.

For a fixed environment, the tag and pending query are determined by the ordinary pair transcript: inspect whether the environment is defined on its last response history. Thus maximal transcripts make the stop convention explicit without changing the all-environments equivalence used in Equation 20.

Let

$$\mathcal{R}_{\text{partial}(B)} = \frac{\text{Prob}(\text{DDS}_{\partial(B)})}{\equiv_{\max}}, \quad (75)$$

where two laws are equivalent when all deterministic environments give the same maximal-transcript law.

Theorem Strict partial random-system algebra. The quotient Equation 75 admits typed renaming, independent parallel composition, and strict finite connection. Maximal-transcript equivalence is a congruence for these operations.

Proof. Measurability of strict connection follows because a fixed visible answer event is the countable union over finite hidden expansion strings of DDS evaluation cylinders. For a finite matching, flatten any nesting of single-edge evaluators. Every order inspects the same unique next DDS answer, feeds it across its matching edge when hidden, and stops at the same first unmatched answer, undefined evaluation, or infinite expansion. This proves connection-order invariance.

For congruence, lift an environment for a connected system to the unconnected boundary. After an answer at either connected port it issues the corresponding hidden query; after another boundary answer it advances the outer environment. A measurable hiding map removes the hidden pairs and maps an infinite hidden tail to strict outer nonresponse. The exceptional preimage is a countable union over the last visible position followed by a countable intersection of hidden-port cylinders, hence Borel. Pathwise equality and pushforward prove connection congruence.

For parallel composition, condition on the deterministic DDS of one factor and simulate its maximal query blocks. A block either reaches a query to the other factor, stops finitely, or continues forever entirely in the fixed factor. In the first case it defines the next value of a deterministic environment for the other factor; in the latter two cases that factor's law is irrelevant. Finite-path cylinders and their countable limits prove measurability. Apply equivalence pointwise and integrate. $\square$

Fixing machine tapes, oracle seed sequences, and initial specification states makes a metered graph deterministic. The selected samplers of Section 15 and finite-step execution make its map into DDS_{∂} measurable. Write $J(G)$ for the resulting maximal-transcript class.

Theorem Route-safe lower realization. At every fixed (κ, b) and boundary B ,

$$J_B : \mathcal{R}_{\kappa, b}^{\text{beh,rs}}(B) \rightarrow \mathcal{R}_{\text{partial}(B)} \quad (76)$$

is an injective homomorphism for renaming, independent parallel, connection, converter action, and identity. Here the domain is the contextual quotient formed from route-safe graphs and route-safe closing contexts. Explicit limited-link nodes remain admitted.

Proof. First prove pointwise full abstraction. If $J(G) \neq J(H)$, some deterministic environment separates their laws on the countable pi-system of finite tagged singletons and finite maximal-transcript cylinders. Along that event it issues only finitely many finite encoded queries. A finite lookup machine with a sufficiently large constant grade implements precisely that test; a semantic score observes system stop without injecting a timeout response. This contradicts behavioral contextual equivalence. Conversely, condition an operational behavioral

context on its tapes, oracle seeds, and initial specification states. Its deterministic one-token execution defines a partial environment plus a semantic score; target transcript equality gives equal score laws, and integration removes the conditioning.

Renaming and tensor are pathwise relabeling and product-sample identities. For connection, couple physical and strict-expanded execution on the same samples. Machine and specification states agree; physical routing takes finitely many hidden steps, cannot exhaust by routing transparency, and has no visible clock. First visible output and strict nonresponse therefore agree pathwise. Converter action is tensor followed by connection, and a bare identity normalizes to the typed target wire. $\square$

Define the route-safe cost quotient analogously using route-safe cost-aware closing contexts. Since every route-safe behavioral context is one such context with a coarser decision map, U restricts to it. Combining the two theorems gives the proved chain

$$\mathcal{R}_{\kappa,b}^{\text{cost,rs}} \xrightarrow{U} \mathcal{R}_{\kappa,b}^{\text{beh,rs}} \xrightarrow{J} \mathcal{R}_{\text{partial}}. \quad (77)$$

Pointwise full abstraction does not put every mathematical environment into the asymptotic efficient test class. It compiles one finite cylinder with a finite, possibly very large constant code. Computational security in Section 16 continues to quantify only over one fixed uniform graded code. Likewise, the pointwise quotient's measurable semantic scores express equality of terminal laws; efficient tests use the charged fixed scorer defined below.

Finite probability common-domain PDSs embed exactly. For a common domain D , truncate any possibly incompatible environment just before its first query outside D . The maximal run of the original environment is a deterministic function of the ordinary compatible run of this truncation: a genuine environment stop becomes EnvStop , while a domain-forced truncation becomes SysStop with its determined pending query. Hence the additional environments neither identify nor separate source-equivalent common-domain PDSs. Comparison with an independently mandated carrier that uses a different nonresponse observation or feedback convention remains a carrier-comparison problem, not a missing operational theorem.

14 Uniform efficient families

14.1 One code, unary parameter

Definition Uniform graded implementation. A uniform graded implementation is one fixed finite normalized network template, with fixed machine codes, codecs, and polynomial grades. Every component receives 1^κ on a read-only parameter tape. Logical party and session identifiers, when needed, are ordinary encoded data.

The template is independent of κ . Unary representation is essential: if κ were supplied in binary, κ^2 machine steps would be exponential in the representation length $\log \kappa$.

The parameter track is not free local storage. For each component, its κ occupied cells and read head are part of the initial native live state and therefore of peak_v , gpeak , and the component space grade; every scan uses ordinary charged transitions. The conventional asymptotic experiment supplies 1^κ during initialization, before the first protocol activation. Work or traffic for physically distributing that initialization is not protocol work in this presentation. If it matters, one must attach a parameter-source/bus resource and charge its copy or shared-read API instead of silently sharing the track.

One fixed universal component can store a polynomial collection of logical sessions on its work tapes. Physical dynamic process creation is therefore not needed for expressiveness at this sequential boundary. It can nevertheless be a convenient presentation.

14.2 Generated networks

A generated presentation is secondary syntax for the primary fixed-template definition. It consists of one fixed deterministic machine G that, on 1^κ , emits a graph code with one fixed external typed boundary B . Implemen-

tation nodes may vary; fixed named specification oracles remain dependencies at B and are not manufactured by the generator.

For $n \in \mathbb{N}$, encode n by the Elias-gamma code of $n + 1$,

$$\text{Nat}(n) = 0^m 1u, \quad m = \lfloor \log_2(n + 1) \rfloor, \quad |\text{Nat}(n)| = 2m + 1. \quad (78)$$

A bit string is length-prefixed and a list is count-prefixed. After a fixed version word, the graph stores a type table, node records, a sorted matching, and a sorted boundary. Each node record contains one program code and indices into fixed codec, grade, and public-owner libraries. The validator rejects every declared count larger than total code length L , out-of-range index, duplicate transition key, ill-typed or repeated endpoint, unsorted record, unconsumed port, or trailing bit. List positions are administrative names: alpha-isomorphic graphs need not serialize identically, and no uncharged graph canonization is assumed.

Compile finite scan/copy and integer-evaluation routine libraries once to literal multitape transitions. Let $c_{\text{dec}}, s_{\text{dec}}, c_{\text{eval}}, s_{\text{eval}}, c_{\text{int}}, s_{\text{int}} \geq 1$ be their six fixed transition and scratch expansion constants. A conservative decoder certificate is

$$\text{DecodeWork}(L) = 128c_{\text{dec}}^2(L+1), \quad (79)$$

$$\text{DecodeSpace}(L) = 16s_{\text{dec}}^2(L+1). \quad (80)$$

Write W for aggregate native component/router work, A for aggregate activations, Q for aggregate named-oracle calls, and let

$$H = 1 + W + A + Q. \quad (81)$$

This bounds the component/router, activation, oracle-access, and final-check slots in a finite metered simulation. Let

$$M = 1 + \kappa + b + \sum_x P_{\text{native}}^{x(\kappa, b)}, \quad (82)$$

where x ranges over every coordinate in the fixed dependency signature, including work, space, randomness, traffic, activation, oracle-call, query/response-bit, public-tariff, and charged oracle-evaluator coordinates. Thus $W, A, Q, H \leq M$, every virtual quota/counter is at most M , and M is polynomial. The universal component stores a directory for the virtual configuration. A length- L code can compactly declare as many as L^2 tape headers, not merely L . In a trace of W native component/router transitions, however, the total number of head moves is at most LW . Headers plus materialized tape intervals, program tables, buffers, stack records, and binary virtual-meter records therefore fit in $O((L + M + 2)^2)$ cells. The use of M , rather than only the number of executed transitions, charges large but mostly unused quotas.

Concretely, directory order gives every node/tape header a positional identity, so a binary index is not repeated beside every virtual tape cell. Each materialized virtual tape is stored as a unary interval over a fixed alphabet with its head marked in place. Its representation grows by at most the number of simulated head moves. There are $O(L)$ virtual meter coordinates—one fixed finite coordinate set per encoded node, edge, and fixed dependency—and their binary records use $O(L \log(M + 1))$ cells. Thus

$$O(L^2 + LW + M + L \log(M + 1)) \subseteq O((L + M + 2)^2), \quad (83)$$

because $W \leq M$. A sparse representation that repeated binary coordinates at every virtual cell would require a different displayed bound and is not the selected interpreter layout.

One fixed master fair tape supplies virtual node i at local random position j with coordinate

$$\text{pair}(i, j) = \frac{(i + j)(i + j + 1)}{2} + j. \quad (84)$$

Cantor pairing is a bijection. On every finite cylinder, distinct virtual coordinates are distinct master coordinates, so the virtual tapes have the independent Bernoulli product law. Reading and caching the required master prefix uses fewer than $(L + M + 2)^2$ random bits. A fixed private grade evaluator and a sequential interpreter have the conservative profiles

$$\text{EvalWork}(L, M) = 128c_{\text{eval}(L+M+2)}^2, \quad (85)$$

$$\text{EvalSpace}(L, M) = 16s_{\text{eval}(L+M+2)}^2, \quad (86)$$

$$\text{InterpretWork}(L, H, M) = 8192c_{\text{int}}H(L + M + 2)^2, \quad (87)$$

$$\text{InterpretSpace}(L, M) = 256s_{\text{int}(L+M+2)}^2. \quad (88)$$

The evaluator receives 1^κ and 1^b on two meter-only read-only administrative tracks. Their total length, scanning, and retained representation are charged inside the M -based work and space bounds; there is no free conversion from a binary magnitude. It constructs the binary quota table with charged integer work and space and never passes the workload, administrative heads, or remaining quotas to a simulated program. The generator receives only 1^κ .

The decoded native ledger charges a separate logical parameter track for every virtual component, as required by the primary definition. The universal implementation may represent their identical immutable contents once together with one virtual head per component; its **physical** space is the displayed interpreter bound, not the virtual native sum. The aggregate native space certificate must nevertheless include all logical parameter tracks, and M includes that coordinate.

Stateful oracle tariffs require one more fixed administrative face. The ordinary reply need not reveal the next public contract coordinate used by the access meter to compute the exact charge. For each fixed named specification dependency, an OracleProxy therefore checks the virtual reservation before a physical seed is touched, makes exactly one call to the unchanged physical occurrence on virtual admission, and receives a private authenticated commit receipt containing the reply/block tag, exact charge, and next public coordinate. It mirrors this into the virtual ledger but gives decoded code only the ordinary reply. Initial public coordinates arrive through the same meter-level face. Receipt codec, validation, traffic, work, and transient space are charged in Equation 88 and the dependency profile. The physical access envelope dominates all virtually admitted calls, so the proxy creates no new exhaustion. This is a fixed accounting adapter, not a generated oracle or a program-visible cost channel. For this invariant, each stateful dependency occurrence is private to the compiled subsystem. Intentional outside sharing instead requires one fixed charged multi-client proxy through which **every** caller passes, so every committed public-coordinate update is mirrored before the next virtual reservation; direct bypass access is not covered.

Theorem Generated-to-fixed compilation. Suppose G has fixed polynomial work and space bounds $g_{\text{work}}, g_{\text{space}}$; every output code is valid, has boundary B , and has length at most $g_{\text{len}(\kappa)}$; and one fixed certificate bounds every **aggregate** native profile coordinate of all decoded components, routers, and fixed specification-access dependencies by $P_{\text{native}}^{x(\kappa, b)}$. Put $L = g_{\text{len}(\kappa)}$, $W = P_{\text{native}}^{\text{work}(\kappa, b)}$, $A = P_{\text{native}}^{\text{act}(\kappa, b)}$, $Q = \sum_j P_{\text{native}}^{\text{calls}_j(\kappa, b)}$, $H = 1 + W + A + Q$, and $M = 1 + \kappa + b + \sum_x P_{\text{native}}^{x(\kappa, b)}$. Then one fixed universal component with its fixed staging and oracle-accounting faces has work profile

$$g_{\text{work}(\kappa)} + 128c_{\text{dec}(L+1)}^2 + 128c_{\text{eval}(L+M+2)}^2 + 8192c_{\text{int}}H(L + M + 2)^2 \quad (89)$$

and space profile

$$g_{\text{space}(\kappa)} + 16s_{\text{dec}(L+1)}^2 + 16s_{\text{eval}(L+M+2)}^2 + 256s_{\text{int}(L+M+2)}^2. \quad (90)$$

A fixed staging face at each port of B retains a first incoming event while initialization determines the virtual receiver and its evaluated capacity. In every route-safe closure—or under a separately supplied external-event envelope for those faces—the compiled graph has, under the master/virtual-tape coupling, the same erased boundary transcript law, virtual terminal owner/status, and exact **virtual native** ledger as the decoded graph. Its physical ledger is only bounded by the displayed transformed component profile plus the ordinary staging/routing and fixed oracle-accounting profiles and need not equal the native ledger. Every fixed stateful dependency occurrence is private to the compiled subsystem, or one declared fixed multi-client accounting proxy mediates all intentional outside callers and mirrors every public-coordinate commit.

Proof. Prefix decoding and validation give Equation 80. Maintain the invariant that decoded code, virtual control states, tapes, heads, buffers, stacks, token owner, specification public coordinates, public ownership labels, meters, and native ledger equal the corresponding native configuration. One sequential interpreter/proxy loop realizes each native component, router, activation, or specification-access action and preserves the invariant. On an oracle call, it first reproduces virtual rejection or makes one identically initialized/seeded physical call; the private commit receipt restores the exact state, public-coordinate, ledger, block, and owner relation without reaching decoded code. Under intentional sharing, the fixed multi-client proxy has mirrored every intervening outside commit; direct callers cannot bypass it. The finite-cylinder pairing argument gives the product random law; induction on at most H action slots gives the same boundary events and virtual status. Summing generator, decoder, grade-evaluator, interpreter, and proxy profiles gives the physical component bounds. On the first external activation, the fixed staging face retains the already routed event while the universal machine performs initialization once. The event's self-delimiting header is sufficient to compute the prospective virtual activation charge. If it fails, the compiler emits the same virtual owner and status without installing the payload. If it fits, its installed length is bounded by the aggregate native space coordinate in M and it is copied once into the virtual configuration. The staging buffer is charged by the final route-safe envelope (or a declared external-event envelope), not falsely bounded by M . No hidden pre-input computation occurs. The decoded state, quota table, and random cache then persist for the system lifetime. $\square$

Per-node bounds are insufficient: a generator could emit too many nodes or grades with parameter-dependent exponents. Likewise, polynomially many logical sessions inside one fixed component require a fixed session program and aggregate profile. A sequential table lookup pays a factor proportional to the number of live sessions on every activation; no unit-cost random access is assumed.

The theorem is a conservative presentation result, not a license to call an arbitrary indexed family uniform or to generate parameter-dependent ideal resources.

14.3 A generated-family calculation

Let $q(\kappa) = \kappa^2 + \kappa + 1$. A fixed deterministic generator emits a chain of $q(\kappa)$ copies of the one-bit program `Flip`; the boundary input enters the first copy and the last copy emits the boundary output. Let ℓ_F be the length of the one fixed valid program code and put

$$c_F = \ell_F + 2\lceil \log_2(\ell_F + 1) \rceil + 10, \quad h_q = \lceil \log_2(2q + 1) \rceil. \quad (91)$$

The constant c_F includes the gamma-prefixed program, one fixed grade index, one fixed public-owner index, and two typed ports. A node index has gamma length at most $2h_q + 1$ and a port index at most three. Accounting for the fixed header, one-entry type table, list prefixes, $q - 1$ internal edges, and two boundary records gives the explicit code bound

$$L_{\text{flip}(\kappa)} \leq c_F q + (q - 1)(4h_q + 8) + 6h_q + 61. \quad (92)$$

A unary nested-loop generator that writes this code has the conservative certificates

$$\text{GenWork} = 128(c_F + 1)(q + 1)^2, \quad \text{GenSpace} = L_{\text{flip}} + 16(q + 1). \quad (93)$$

The space expression retains the generated code; counting only loop counters would be false because $L_{\text{flip}} = \Theta(q \log q)$. Its second term also covers the generator's own unary parameter track and head, since $q = \kappa^2 + \kappa + 1 \geq \kappa$. One `Flip` activation uses four steps and each one-bit internal route uses five, so one boundary query has

$$\text{NativeWork} = 9q - 5, \quad \text{NativeActs} = q, \quad \text{NativeTraffic} = q - 1, \quad \text{NativeSpace} \leq q(\kappa + 3). \quad (94)$$

It returns $x \text{ xor } (q \bmod 2)$. For this one-query calculation at $b = 1$, put $M_F = \kappa + q(\kappa + 16) + 16$, which dominates one plus every displayed aggregate coordinate, including the q logical unary parameter tracks. Here $H = 1 + (9q - 5) + q = 10q - 4$. Substituting $L = L_{\text{flip}}$, this H , and $M = M_F$ into Equation 80 and Equation 88 gives

$$\begin{aligned}
\text{FixedWork} &\leq 128(c_F + 1)(q + 1)^2 + 128c_{\text{dec}(L+1)}^2 \\
&\quad + 128c_{\text{eval}(L+M_F+2)}^2 \\
&\quad + 8192c_{\text{int}(10q-4)}(L + M_F + 2)^2,
\end{aligned} \tag{95}$$

$$\begin{aligned}
\text{FixedSpace} &\leq L + 16(q + 1) + 16s_{\text{dec}(L+1)}^2 \\
&\quad + 16s_{\text{eval}(L+M_F+2)}^2 + 256s_{\text{int}(L+M_F+2)}^2.
\end{aligned} \tag{96}$$

The fixed interpreter has the same erased transcript and virtual native ledger, but its physical ledger is Equation 96, not Equation 94. This example separates the four facts that are often compressed into the word “uniform”: one generator code, an explicit generated-code bound, an aggregate native profile, and a fixed interpreter profile.

14.4 Nonuniformity and auxiliary input

Counterexample Small members do not make a uniform family. Let $H \subseteq \mathbb{N}$ be nonrecursive. At parameter κ , hard-code the bit $1[\kappa \in H]$ in a one-state constant-time machine. Every member has a finite efficient implementation, but no uniform generator implements the family.

A nonuniform implementation is instead a fixed template with an advice string z_κ satisfying $|z_\kappa| \leq p(\kappa)$ for one fixed polynomial. The advice sequence and bound are part of the object.

There are three related test classes.

- **Pure uniform:** one fixed test code and no arbitrary advice; any public input is produced by a fixed uniform generator.
- **Auxiliary-input:** the test code is fixed, but security is uniform over all a with $|a| \leq q(\kappa)$. Choosing a worst a_κ at every parameter gives this notion nonuniform strength.
- **Explicit nonuniform:** polynomial advice or circuit families are admitted directly.

Calling all three “uniform PPT” conceals a quantifier that can change a theorem. We state the mode whenever computational indistinguishability is used.

14.5 Efficient contexts and ambient policies

A pure uniform efficient context consists of a fixed graded test network, a fixed deterministic graded input generator G_D , a fixed uniform terminal-scorer code, and a fixed monotone polynomial p_D . The generator receives only 1^κ , has fixed polynomial work, space, and output-length grades in κ , and cannot inspect b . Its execution, retained output, and installation of that output as the test’s public input are included in the test initialization profile. Its projection of its own terminal configuration is either the canonical finite encoding of declared context-owned coordinates or one fixed uniform effective code; that projection’s output length, work, and space belong to the test profile just like the public-ledger projection. The scorer is run after the interaction on the resulting self-delimiting terminal record. A fixed default bit is used if that postprocessing itself blocks or exhausts. At parameter κ and generated public input a , the context selects

$$b_D = p_{D(\kappa+|a|)}. \tag{97}$$

Every participating fixed component uses its own polynomial transformer of (κ, b_D) . By the fixed-graph theorem, the closed execution has a polynomial global bound whose degree may depend on the fixed context but never on κ .

For behavioral indistinguishability, every such metered test is admissible: even a self-exhausting interaction has a total decision through the fixed scorer and default. The scorer sees only the erased terminal record and never creates a wire response. Availability uses the smaller completion class $\mathcal{C}$ from Section 12, whose members certify their own safety and conditional progress under an interface envelope. This distinction prevents a malicious test from converting its own exhaustion, block, or silent stop into a false resource availability counterexample.

This is a metered form of the environment-relative quantifier used in reactive runtime. Hofheinz, Unruh, and Mueller-Quade call a stronger variant **uniform reactive polynomial time** when one polynomial transforms a bound on the context’s runtime [11, Def. 33]. The analogy concerns only cost quantifiers. We import neither their UC security definition nor its machine roles.

14.6 Evidence from reactive runtime

The literature distinguishes several useful but incompatible notions:

Notion	Bound	Connection behavior
Lifetime PPT	One polynomial in κ per machine	Closed for fixed hard clocks; poor for open services
Per activation	Polynomial in current input	Message growth can be exponential
Polynomial shape	Output and activations polynomial in outside input	Requires a composition check
Reactive polynomial	Every strict-PPT context gives an almost-polynomial run	Not closed under arbitrary connection
IITM environmental	Every universal environment gives an almost-bounded run	Under time-lock puzzles, almost-bounded components need not compose
Metered grade	Strict polynomial quota in (κ, b)	Closed as bounded partial behavior

Hofheinz, Mueller-Quade, and Unruh relate work to polynomial prefixes and outside traffic, and explicitly show that polynomially shaped components can form an internal loop [10, Defs. 3.1 and 4.1]. Their later reactive polynomial-time formulation retains the same nonclosure and places a runtime premise on the composed protocol [11, Defs. 9-10].

The IITM model distinguishes universally bounded environments, environmentally strictly bounded systems, and environmentally almost bounded systems [12, Defs. 7-10]. Under time-lock puzzles, two almost-bounded systems can generate one another’s rare bad inputs and cease to be almost bounded [12, Sec. 8.2]. This rules out negligible runtime overrun as an unconditional wiring discipline.

Our meters make every rare overrun a lower exhausting branch. They are not the “strong reactive polynomial-time” semantic simulator-validity predicate criticized in that literature: every protocol and simulator here is a syntactically graded code, and exhaustion contributes to the compared lower behavior. A performance theorem may later prove that its probability is zero or negligible.

15 Abstract specifications as charged oracles

15.1 Efficient access is not implementation

An ideal resource need not have a finite program. A random oracle represents a random function table; an ideal channel may abstract an unmodeled physical service; an abstract kernel may use a noncomputable real probability. These objects can still have finite encoded query and response interfaces.

Definition Two lower sorts. An **implementation** has a finite machine code, named random tapes, an exact ledger, and grades. A **specification oracle** is a family of abstract random systems with measurable conditional behavior, effective wire encodings, and an access tariff. Only the former is called efficiently implementable.

An oracle is **efficiently accessible** on a workload when parsing its wire values and paying its tariff require polynomial resources. No internal implementation conclusion follows. This separation permits computational experiments relative to ideal resources without placing them artificially in the machine image.

Definition Finite dependency signature. A dependency signature Γ is a fixed finite list of oracle packages. Each package fixes its typed interface, initialization rule, conditional kernel, effective codecs, public-state coordinate, reservation and charge functions, evaluator profiles, and independence convention. A Γ -relative graph has finitely many statically named oracle occurrences, each an instance of a package in Γ .

Before independent parallel composition, occurrence names are made disjoint. Distinct occurrences receive distinct initial-state and seed coordinates; intentional sharing uses one occurrence with several callers. Absorbing a graph into a test retains these very occurrences and coordinates—it neither clones them nor resamples their initial state.

The finite list of **packages** does not impose one universal constant on the number of oracle calls. Calls are bounded by the occurrence grades. Nor does it permit a generated graph to manufacture a parameter-dependent number of independent oracle instances. Such an interface must be supplied as one fixed indexed product-state package with its own effective multiplexer, reservation, aggregate tariff, and initialization law. Relative closure below is always with respect to one declared Γ ; enlarging Γ changes the model and is not a proof step.

15.2 Tariffs

Fix a named oracle occurrence O . Its query and response sets Q_O, R_O are countable and have fixed effective prefix-free encodings. The hidden state space S_O is standard Borel. The specification is a measurable kernel

$$K_O : S_O \times Q_O \rightarrow \mathcal{P}((S_O \times R_O \times \{\text{reply}\}) + (S_O \times E_O \times \{\text{block}\})), \quad (98)$$

where E_O is a countable finite-code set of block reasons. A block may change the hidden state, but it does not place an ordinary error value on an external wire.

For pathwise arguments, select a measurable randomization

$$\text{sample}_O : S_O \times Q_O \times [0, 1] \rightarrow (S_O \times R_O \times \{\text{reply}\}) + (S_O \times E_O \times \{\text{block}\}) \quad (99)$$

whose last-coordinate pushforward is Equation 98. Such a selection exists for standard-Borel kernels by the randomization lemma [13, Lemma 3.22]. Each occurrence owns a named iid sequence of uniform seeds. Two occurrences with the same kernel name are independent after alpha-renaming; shared state is represented by one occurrence with several callers.

The access meter sees no arbitrary hidden state. Instead the contract exposes a countable, effectively encoded public coordinate $\text{pub}_O : S_O \rightarrow U_O$. For $z = (s', r, \text{reply})$ or (s', e, block) , let $\bar{z}$ retain only the tag, the finite response/block code, and $\text{pub}_{O(s')}$; it never exposes s' . The contract fixes both an exact charge

$$\text{charge}_{O(\kappa, b, u, q, \bar{z})} \in \mathbb{N}^d \quad (100)$$

and a finite pre-call reservation

$$\text{reserve}_{O(\kappa, b, u, q)} \in \mathbb{N}^d. \quad (101)$$

The charge coordinates include at least one oracle call, encoded query and response traffic, and any declared public tariff. They dominate the corresponding code lengths. Canonical router work and edge traffic are charged by the ordinary ledger; an API-specific access-routing coordinate may be added, but the same transfer is never counted twice within one declared profile. The reservation has the **strong envelope** property

$$\text{charge}_{O(\kappa, b, u, q, \overline{\text{sample}_{O(s, q, v)}})} \leq \text{reserve}_{O(\kappa, b, u, q)} \quad (102)$$

for every compatible hidden state s and every $v \in [0, 1]$, not merely almost every seed. Its evaluator and the charge evaluator are effective fixed codes with declared polynomial profiles. Their work is charged either to the caller or to an explicit administrative component. The next public coordinate is maintained by the contract, so neither evaluator traverses hidden s or s' . The kernel itself may remain noncomputable.

Evaluator charging has its own atomicity discipline. The deterministic reservation evaluator runs before any fresh seed is touched; if its declared public work/space envelope does not fit, the call exhausts without sampling or changing specification state. Its result is then checked together with a public envelope for every **post-sample** charge-evaluator, record-construction, and commit action. Only after this combined reservation

fits may the kernel be sampled. Thus no effective evaluator can introduce a response-dependent exhaustion between sampling and the atomic state/charge commit. The exact evaluator work is retained in the ledger and unused reserved work is released.

Definition Atomic oracle admission. Let B be the remaining access-budget vector and j the occurrence's next seed index. First run the fixed reservation evaluator under its public pre-sample envelope. If that evaluation or the combined semantic and post-sample administrative reservation does not fit in B , execution returns owner-labeled Exhaust without evaluating Equation 99, consuming the seed, changing the oracle state, or revealing a response-dependent charge. If it fits, execution reads the seed once, runs Equation 99 once, runs the now-funded charge evaluator on that outcome, commits the next state once, records the exact semantic and evaluator charges, and either delivers the response or terminates in semantic Block. The unused reservation is released inside the external meter and is not visible to programs.

This ordering is essential. Sampling first and then retaining only responses whose exact charge fits would condition the response distribution on its cost. Rolling back and trying again would additionally hide the exhaustion event. Pre-reservation preserves the advertised kernel on every admitted call. An earlier response may still influence whether a later call fits; that is honest response-dependent future cost, not selection of the current response.

Counterexample Oracle laundering. Suppose a one-bit query returns an exponentially long witness or the answer to a nonrecursive predicate. If a call counts as one ordinary Turing step and response delivery is free, a caller appears efficient while importing unbounded hidden computation. Separate call, response-traffic, and tariff coordinates prevent this inference.

The internal work of O remains unchanged by definition. To price it, select an implementation of O or reify a processor resource.

15.3 Oracle-relative probability

Machine tapes and named oracle seed sequences are independent unless a common resource explicitly correlates them. The atomic rule gives four facts needed by the lower algebra.

Theorem Admitted-kernel and budget-monotonicity theorem. Condition on a complete pre-call history that fixes s, q, B , and the seed index. On the event that Equation 101 fits, the conditional next-state and response law is exactly $K_{O(s,q)}$. Moreover, couple two experiments using the same machine tapes and named seed sequences and give one a larger coordinatewise budget. Every call admitted by the smaller experiment is admitted by the larger one and produces the same outcome and exact charge.

Proof. Reservation evaluation and admission are decided from public pre-call data before the fresh seed is read. Conditional on the history, that seed remains uniform and Equation 99 therefore has pushforward $K_{O(s,q)}$. If the combined reservation fits below $B \leq B'$, both executions admit. Their seed indices agree, so the selected sampler, state update, evaluator trace, and exact charge agree. Induction gives identical traces up to the smaller experiment's first exhaustion. $\square$

Theorem Finite terminal-kernel theorem. Fix a closed finite normalized graph, finite evaluated machine and router quotas, finite oracle-call quotas, and bounded-call contracts satisfying Equation 102. Then there is a unique measurable law on

$$\{\text{Success}\}(\tau, o, \ell) + \{\text{Block}\}(v, \tau, o, \ell) + \{\text{Exhaust}\}(v, \tau, o, \ell). \quad (103)$$

The law is measurable in the encoded public input and the initial oracle state.

Proof. The machine configuration space is countable. Its finite product with the standard-Borel oracle states is standard Borel. Machine and router steps are deterministic measurable kernels. An oracle step is a measurable piecewise kernel: a Dirac exhaustion kernel on rejection, and the pushforward of Equation 98 through the commit-and-deliver map on admission. Every nonterminal implementation step consumes positive work, and

every admitted oracle step consumes a positive call coordinate. Receiver-delivery and commit bookkeeping that does not itself increment either coordinate occurs only a fixed finite number of times between those charged actions. Therefore the finite evaluated quotas give a deterministic finite horizon. Make terminal states absorbing, compose to that horizon, and push forward through the displayed result map. $\square$

Theorem No budget minting. An admitted oracle call never increases local work, local-space capacity, or any access-budget coordinate. Every delivered response bit is charged; reading, copying, storing, parsing, or re-emitting it consumes ordinary implementation resources. Thus Q admitted calls and response-bit quota R import at most Q oracle outcomes and R encoded response bits.

The theorem says nothing about the information or hidden computational power of an answer. It says only that this power cannot be laundered into unrecorded machine work. An oracle is **efficiently accessible** on a declared query domain when reachable public-coordinate and query lengths, reservations, and codec/reservation/charge-evaluator work—including the pre-sample reserve-evaluator envelope and the post-sample evaluator/commit reservation—have fixed polynomial envelopes in (κ, b) , and the domain is closed under admitted public-coordinate updates. This remains distinct from efficient implementability.

The finite terminal-kernel theorem is sufficient for every fixed budgeted security experiment. Taking the product of the named seed sequences and initial specification-state laws also gives the measurable lifetime partial-DDS law used by Section 13; its strict-feedback congruence has already been proved there. A carrier with a different nonresponse convention requires a separate comparison.

15.4 Responses without a finite envelope

If one public query has encoded responses of unbounded support, no finite strong reservation can dominate it. There are two honest refinements. A public cap with an explicit overflow response defines a different bounded specification. Alternatively, expose a stateful streaming interface

$$\{\text{Start}\}(q) \rightarrow h, \quad \{\text{Next}\}(h) \rightarrow \{\text{Chunk}\}(w, \text{done}), \quad (104)$$

with a fixed maximum chunk length c . **Start** samples the original response and next state once, commits that state, and stores the unrevealed suffix behind a handle whose public code has a fixed contract bound. Each admitted **Next** call commits one charged chunk; a rejected call reveals nothing and leaves the handle unchanged.

Attach a fixed metered reassembly converter A_c . In the single-token model it retains the token, permits one active handle, stores chunks in ordinary metered memory, and emits one response only after reassembly. The chunk interface is internal.

Theorem Streaming/reassembly coupling. Couple the original one-shot reference kernel and the streaming specification by the same sampled response. Whenever the converter has enough chunk calls, response traffic, work, output work, and buffer space, the reassembled response and committed next state agree exactly. After hiding the chunk interface, the total-variation distance between their finite erased transcript laws is at most the probability that one of those resources is insufficient.

Proof. Induction on the chunk index shows that the delivered concatenation is always the corresponding prefix of the one sampled encoding. On completion the concatenation, output, and state agree. Under the common-response coupling, disagreement is contained in the stated exhaustion event; the coupling inequality gives the bound. Conditioning on successful reassembly is not used and would generally favor shorter responses. $\square$

15.5 A complete unbounded-response instance

Define the stateless oracle **GeoBits** to sample $L \in \mathbb{N}$ with

$$P[L = \ell] = 2^{-\ell-1}, \quad (105)$$

then sample U uniformly in $\{0, 1\}^L$ and return (L, U) . Use the prefix code $1^L 0 U$, of length $2L + 1$. This response is finite almost surely but has no finite strong atomic envelope.

Let the streaming oracle store this one sampled code behind its single active handle. On **Next** it returns at most

$$c(\kappa) = 2\kappa \quad (106)$$

code bits, with the last full or partial chunk tagged done. Let the reassembler have

$$Q(\kappa, b) = b + 1 \quad (107)$$

chunk calls. Fixed compiled copy routines give constants $\alpha_0, \alpha_1, \alpha_2, \beta_0, \beta_1$ for which it is sufficient to fund

$$\text{Calls} \leq Q + 1, \quad \text{ChunkBits} \leq Q(c + h), \quad (108)$$

$$\text{Work} \leq \alpha_0 + \alpha_1 Q + \alpha_2 cQ, \quad \text{Space} \leq \beta_0 + \beta_1 cQ, \quad (109)$$

where h is the fixed handle/chunk-tag overhead. Output traffic is at most cQ plus its fixed outer tag. The fixed β_i are chosen to cover the reassembler's retained unary parameter track and head as well as its chunk buffer: since $\kappa \geq 1$, $Q \geq 1$, and $cQ = 2\kappa Q$, this storage is dominated by the displayed term. The unbounded sampled suffix remains state of the specification oracle; it is not falsely charged as polynomial implementation memory.

Proposition Concrete geometric-streaming loss. For $\kappa \geq 1$, fund Equation 109 and hide the handle/chunk interface. The reassembled oracle and the one-shot GeoBits reference have finite-transcript total-variation distance at most

$$2^{-\kappa(b+1)}. \quad (110)$$

All implementation-side calls, traffic, work, output work, and space are bounded by fixed polynomials in (κ, b) .

Proof. Reassembly fits whenever $2L + 1 \leq cQ = 2\kappa(b + 1)$, equivalently $L \leq \kappa(b + 1) - 1$. The geometric tail is

$$P[L \geq \kappa(b + 1)] = 2^{-\kappa(b+1)}. \quad (111)$$

On its complement the streaming coupling returns the identical prefix code and hence the identical pair (L, U) . The funded local routines cannot be the first failure, so Equation 110 follows from the coupling theorem. Since c and Q are fixed polynomials, every coordinate in Equation 109 is polynomial. In particular the bound is at most $2^{-\kappa}$ for every public workload b . $\square$

The example does not condition the oracle on producing a short answer. `Start` commits the same geometric sample in both experiments; the long-answer event remains an explicit exhaustion tail. Nor does the example assert that the oracle's hidden sampling work is supplied by the caller.

15.6 The random-oracle example

Let $\text{RO}_{m,n}$ be a stateful ideal random function from m bits to n bits. Its access charge includes at least one call, m query bits, and n response bits; ordinary edge routing is recorded separately. Its conceptual table is not a PPT implementation.

A concrete lazy-sampling dictionary is a different object. It may realize every polynomial-query transcript of $\text{RO}_{m,n}$, while its dictionary time, space, and random-bit use appear in the implementation ledger. The two-sort model can express both statements without identifying them.

15.7 A bounded-query random-function realization

The preceding claim can be made exact for finite-length random functions. Let $m = m(\kappa)$ and $n = n(\kappa)$ be fixed effectively evaluated polynomial length functions, and let $Q = Q(\kappa, b)$ be a fixed polynomial call envelope. One fixed component `LazyRF` maintains a sequential list of distinct pairs (x, y) . On an m -bit query it scans the list. If x is present, it copies the stored n -bit value. Otherwise it reads the next n bits of its named random tape, appends (x, y) , and returns y . It never reads Q or its remaining meter; Q only grades the closed run.

With the elementary bit routines used below, the conservative lifetime profile

$$\text{Work}_{\text{LRF}} \leq Q(4Q(m + n + 8) + 8(m + n + 1)) + \text{LenEval}(\kappa), \quad (112)$$

$$\text{Space}_{\text{LRF}} \leq (Q + 1)(m + n + 8) + \text{LenSpace}(\kappa), \quad (113)$$

$$\text{Random}_{\text{LRF}} \leq Qn, \quad \text{Acts}_{\text{LRF}} \leq Q \quad (114)$$

Here LenEval is the work of the fixed machines that evaluate and validate $m(\kappa)$ and $n(\kappa)$. The corresponding LenSpace includes their simultaneous scratch and LazyRF's retained logical 1^κ parameter track and head. Thus Equation 114 does not obtain its asymptotic input or its length computations as free storage.

The work bound also pays for repeated-query scans, validation, appending, and output construction. Boundary query/response traffic contributes Qm and Qn when a closing context is attached.

Proposition Exact bounded-query random function. Fund LazyRF by Equation 114. Also fund boundary routing for at most Qm query bits and Qn response bits, either by a route-safe closure or by an explicit envelope. On every adaptive history of at most Q valid calls, its erased response law is exactly that of a uniformly sampled function $\{0, 1\}^m \rightarrow \{0, 1\}^n$. The LazyRF occurrence never exhausts and answers every delivered query on this domain. A closed productivity claim additionally uses the completion context's response envelope and adds its failure bound χ_D ; it is strong when $\chi_D = 0$. The implementation uses no specification-oracle calls.

Proof. Induct over the adaptive query history. A repeated input returns the unique stored value in both experiments. Conditional on every preceding transcript, the value of a new input under a uniform random function is an independent uniform n -bit string; the next unused block of the named fair tape has the same conditional law. Thus all finite adaptive transcript laws agree.

Before the i th call the list has at most Q records. Each encoded record has length at most $m + n + 8$; a sequential scan, including passage over stored values on a miss, costs at most $4Q(m + n + 8)$ bit steps. Validation, a worst-case append, sampling, and constructing the blank output register fit within $8(m + n + 1)$ further steps. Summing Q calls proves the work bound; at most Q records plus one active input/output region prove the space bound. At most one fresh n -bit block is used per call. The displayed meters therefore never fire. The route premise separately funds delivery and return. The completion context definition then contributes exactly its already declared χ_D term to any closed-success statement. $\square$

This is a bounded-lifetime realization theorem, not a finite implementation of an unbounded random oracle table. Increasing the workload supplies a larger polynomial dictionary profile. A pseudorandom replacement would be a computational construction with a separate assumption and error; none is silently used here.

16 Computational tests and reductions

16.1 Costed and behavioral distinguishers

A cost-aware computational distinguisher is a fixed uniform graded network with one challenge boundary, together with one fixed uniform graded terminal-scorer program. At fixed (κ, b) , the lower closed experiment terminates in

$$\{\text{Success}\}(\tau, o, \ell), \quad \{\text{Block}\}(v, \tau, o, \ell), \quad \text{or} \quad \{\text{Exhaust}\}(v, \tau, o, \ell). \quad (115)$$

The scorer receives a self-delimiting encoding of the permitted terminal record after interaction has ended. Its transitions, input reading, work, space, and record processing are included in Profile_D ; if it blocks or exhausts, a fixed one-bit default completes the experiment. A cost-aware scorer may use the maximal transcript, its own final observation, the public failure owner, and the exact report. A behavioral scorer receives only the image under U .

For a uniform test, both (i) the projection of the test's own terminal configuration to o and (ii) the public-ledger report projection are fixed uniform effective codes, or canonical encodings of declared accessible coordinates. Each has a polynomial output-length and evaluation certificate. Terminal-record construction is charged together with scoring. The test profile therefore includes a bound on all record bits in addition to execution coordinates. An arbitrary measurable or exponentially expanding terminal-state or report map remains meaningful in the pointwise quotient but is not an efficient observer.

The encoded report supplied to the scorer is frozen at the end of the interaction phase. Scorer and report-projection work is charged for admissibility but is not recursively inserted into the report being read. An outer audit may record that postprocessing cost after the decision; the decision experiment itself has no self-referential cost input.

This restriction is specific to the computational class. The pointwise contextual quotient in Section 12 legitimately quantifies over every measurable terminal bit map in order to identify full outcome laws. Giving such an arbitrary map for free to a purported efficient test would smuggle noncomputable advice into the distinguisher. The graded scorer prevents that mistake. Its post-run invocation is still a meta-level observation of a maximal semantic run, not a program-visible timeout: it creates no resource response and no successor round.

For compatible lower resources R, S , define

$$\text{Adv}_{D,b}^{(\kappa,a)}(R, S) = |P[D^{b(R;\kappa,a)} = 1] - P[D^{b(S;\kappa,a)} = 1]|. \quad (116)$$

For a declared input mode M , let $\mathcal{A}_{D,M}(\kappa)$ be the singleton containing $G_{D(1\kappa)}$ in pure uniform mode, all strings of length at most $q_{D(\kappa)}$ in bounded auxiliary-input mode, or the singleton containing the named generated input and advice in explicit nonuniform mode. Put

$$\text{Adv}_{D,p_D;M}^{\kappa(R,S)} = \sup_{a \in \mathcal{A}_{D,M}(\kappa)} \text{Adv}_{D,b_{D(\kappa,a)}}^{(\kappa,a)}(R, S), \quad b_{D(\kappa,a)} = p_{D(\kappa+|a|)}. \quad (117)$$

Consequently a behavioral distinguisher cannot distinguish blocking from exhaustion with the same visible prefix and closing observation, and cannot attribute the nonresponse to one owner. This restriction concerns the observation algebra, not the set of executions.

A function $\nu : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is negligible if for every $c > 0$ it is eventually below κ^{-c} . Computational indistinguishability in the declared mode M means

$$\forall D, p_D \quad \exists \nu_{D,p_D} \text{ negligible} \quad \text{Adv}_{D,p_D;M}^{\kappa(R,S)} \leq \nu_{D,p_D}(\kappa), \quad (118)$$

where D is one fixed uniform code and p_D one fixed polynomial ambient policy. The negligible function may depend on that fixed test; taking a pointwise supremum over all polynomial-time codes would change the quantifier and is not intended. In pure mode the supremum in Equation 117 is over one polynomially graded generated input, whose initialization profile is part of D . In bounded auxiliary-input mode it is $\sup_{|a| \leq q_{D(\kappa)}}$, which has nonuniform advice strength. The explicit nonuniform variant admits and names the input/advice sequence together with one fixed polynomial combined-length bound. Supplied auxiliary input or advice does not charge a generator, but its occupied cells, validation, delivery, and subsequent processing remain in the test profile.

16.2 Graded converters

A converter code has outer and inner typed ports and a polynomial **profile transformer**

$$T_\alpha : \text{Profile}_{\text{outer}} \rightarrow \text{Profile}_{\text{absorbed}}. \quad (119)$$

The profile contains time, peak space, random bits, traffic, activations, named oracle calls, terminal-record length, and report/scorer work. It is an upper-bound transformer, not the exact ledger. Sequential converter composition combines transformers by polynomial substitution and addition; finite parallel composition combines their coordinates.

At fixed (κ, b) , a metered converter has a finite bound on hidden calls uniformly over all inner answers. It therefore satisfies the finite inner-query discipline of Jost's discrete converter definition [8, Def. 2.2.2], with a bound that may depend on the parameter and workload.

16.3 Exact absorption

Theorem Converter and parallel-resource absorption with profile reindexing. Let D interact with αR . Let $D[\alpha]$ be the network obtained by placing the same converter occurrence between D and its challenge port. Also fix a feasible resource graph S , and let $D[\cdot \parallel S]$ be the test obtained from the closed experiment

$D(R \parallel S)$ by regarding only the R -ports as its challenge boundary. All graphs are relative to the same finite dependency signature Γ , and names are made disjoint before parallel composition.

For every fixed parameter, ambient workload, tapes, selected oracle seed sequences, and initial specification states, the two closed executions in each pair have the same decision: $D(\alpha R)$ versus $D[\alpha](R)$, and $D(R \parallel S)$ versus $D[\cdot \parallel S](R)$. Moreover, for fixed polynomial profile transformers T_α and T_S accounting for the absorbed graphs,

$$\text{Profile}_{D[\alpha]} \leq T_{\alpha(\text{Profile}_D)}, \quad \text{Profile}_{D[\cdot \parallel S]} \leq T_{S(\text{Profile}_D)}. \quad (120)$$

Proof. In either pair, the two descriptions normalize to the same typed graph with the same component codes, random tapes, meters, specification occurrences, initial states, oracle seeds, and public ownership labels. In the parallel-resource case, alpha-renaming is performed once before both descriptions and the occurrences of S are retained, not re-instantiated. Absorption does not relabel protocol work as context work. In the route-safe regime the identical final aggregate also derives the identical canonical-router envelope. Hence the labeled traces and decisions are pathwise equal. The two profile inequalities are the grade certificates for those normalized graphs. $\square$

The ambient index b need not change; the certified aggregate profile does. Taking suprema over tests admitted by a profile gives

$$\Delta_{P(\alpha R, \alpha S)} \leq \Delta_{T_{\alpha(P)}}(R, S). \quad (121)$$

Same-profile nonexpansion is claimed only when a separate domination proof shows $T_{\alpha(P)} \leq P$.

A parallel resource may contain ordinary code as well as specification occurrences. Both remain in the absorbed test. Every occurrence retains its kernel, initialization, seed coordinate, tariff, and call profile, while its implementation nodes retain all ordinary ledger coordinates. This is the operational cost form of the reduction reindexing in Jost's relaxation theorem [8, Thm. 2.2.11].

16.4 The asymptotic quotient

If $P(\kappa)$ is polynomial and T_α is a fixed multivariate polynomial, then $T_{\alpha(P(\kappa))}$ is polynomial. Efficient tests are therefore closed under absorption. Parallel specifications are subject to the same uniformity rule as implementation graphs: a fixed finite collection of named occurrences is harmless; a parameter-dependent collection requires one fixed indexed product-state specification with an effective multiplexer, reservation, and aggregate profile. The generated-network compiler creates implementation nodes and cannot manufacture ideal independence.

Theorem Asymptotic nonexpansion. Computational indistinguishability for uniform polynomial-workload tests is preserved by every fixed uniform polynomially graded converter and every fixed feasible parallel resource relative to the declared dependency signature. Sequential construction composes profile transformers and adds negligible errors. A parameter-dependent hybrid sum is allowed when one fixed uniform hybrid generator emits all intermediate experiments, their number has a fixed polynomial bound, one aggregate polynomial profile bounds them all, and one negligible function uniformly bounds every adjacent gap after the displayed reindexing.

Proof. Absorb the converter or parallel resource using the exact graph equality. Polynomial substitution keeps the new test efficient. Apply the assumed negligible bound to that test. Sequential composition uses the triangle inequality. For generated hybrids $H_0^\kappa, \dots, H_{h(\kappa)}^\kappa$, one fixed code supplies every adjacent distinguisher and a polynomial $p(\kappa)$ bounds both h and the aggregate profile. If a negligible ν bounds every adjacent gap, the total is at most $p\nu$, which is negligible. Merely choosing a new family of hybrid descriptions separately for each κ would be nonuniform and is not covered. $\square$

Counterexample Elementwise efficiency is not specification-uniform. For each fixed $n \in \mathbb{N}$, let

$$\eta_{n(\kappa)} = 1[\kappa = n]. \quad (122)$$

Every η_n is negligible because it has finite support. It is realized by a fixed uniform machine family whose exceptional branch compares its unary parameter with the hard-coded constant n . Nevertheless

$$\sup_{n \in \mathbb{N}} \eta_n(\kappa) = 1 \quad (123)$$

for every κ . Thus a set of individually uniform efficient resource families does not, merely by membership, supply a negligible bound uniform over the specification.

This is the quantifier made explicit by Jost’s parallel relaxation: $\varepsilon^S(D) = \sup_{S \in \mathcal{S}} \varepsilon(D[\cdot \parallel S])$ [8, Thm. 2.2.11]. Pointwise absorption proves that each displayed test is feasible. It does not move the supremum through the negligibility quantifier.

Definition Uniform specification certificate. A specification $\mathcal{S}$ is **uniformly presented for a selected input mode** if it has one fixed presentation mechanism, one fixed polynomial descriptor bound, one aggregate polynomial profile, and the declared dependency signature Γ . For implementations the mechanism is one uniform selector/compiler taking a descriptor z . For ideal behavior it is one fixed indexed specification package with an effective multiplexer, reservation, aggregate tariff, and initialization rule. The selected input mode quantifies over all allowed z outside the probability.

A construction or relaxation witness for $\mathcal{S}$ must additionally supply one negligible ν_D for each fixed admitted test and policy **after** taking the supremum over all allowed descriptors. Equivalently, it may supply a concrete bound uniform over the aggregate profile whose substitution is negligible.

Theorem Specification-level lifting rule. Converter and parallel-resource absorption lift from individual families to a specification when the relevant error witness is uniform in the sense above. A fixed finite specification is the special case obtained by taking the maximum of finitely many negligible bounds. Sequential and parallel construction of uniformly presented specifications remains valid when the composed selector, descriptor bound, dependency signature, aggregate profile, and uniform error envelope are supplied.

Proof. The exact absorption theorem gives the required equality separately for every descriptor. The uniform presentation turns the family of absorbed experiments into one admitted test with the descriptor quantified according to the selected mode; its aggregate transformer is polynomial. The assumed ν_D bounds the supremum, so asymptotic nonexpansion applies. In the finite case, a finite maximum is bounded by the finite sum of the corresponding negligible functions. Sequential composition composes the two uniform presentation mechanisms and profile transformers, and the triangle inequality adds their uniform errors. Without the uniform witness, the preceding diagonal example blocks the conclusion. $\square$

This theorem is the content hidden by the phrase “PPT is closed.” In concrete security, Equation 119 and the oracle-query profile must be reported. An asymptotically polynomial but practically enormous simulator can make a reduction useless.

16.5 A complete reduction calculation

The following example keeps both the information-theoretic loss and the machine overhead visible. Let $m = s + n$ and let RF_m and RP_m be, respectively, a uniformly sampled function and permutation on m bits. For a fixed s -bit tag j and $\ell \leq m$, define

$$F_j = \text{Trunc}_\ell \circ \text{RF}_m \circ \text{Tag}_j, \quad P_j = \text{Trunc}_\ell \circ \text{RP}_m \circ \text{Tag}_j, \quad (124)$$

where $\text{Tag}_{j(x)} = j \parallel x$ and Trunc_ℓ returns the first ℓ bits. Fix the bit-machine routines $\text{WriteConst}(\text{L})$, $\text{Copy}(\text{L})$, $\text{ReadFixed}(\text{L})$, and $\text{Route}(\text{L})$ with respective work bounds $2L + 1$, $3L + 1$, $2L + 1$, and $2L + 3$; emitting an event costs one step.

One outer query then adds

$$C_{\text{wrap}(s,n,\ell)} = (2s + 3n + 3) + 2(2m + 3) + (2m + 3\ell + 3) = 8s + 9n + 3\ell + 12 \quad (125)$$

work units. If D has profile (t, a, c, S, r) and makes at most q outer queries, absorption gives

$$\text{Work} \leq t + qC_{\text{wrap}}, \quad \text{Acts} \leq a + 2q, \quad \text{Traffic} \leq c + 2qm, \quad (126)$$

$$\text{Space} \leq S + 4m + 2\kappa + 10, \quad \text{Calls} \leq q, \quad \text{QBits, RBits} \leq qm, \quad \text{Random} = r. \quad (127)$$

The deliberately loose space term covers source, destination, routing, and active wrapper buffers together with the two wrappers' logical 1^κ tracks and heads. The wrappers themselves use no randomness.

Couple lazy sampling of the function and permutation so that they return the same fresh output until the function sampler repeats an earlier output. For d distinct queried inputs the exact disagreement probability is

$$1 - \prod_{i=0}^{d-1} \left(1 - \frac{i}{2^m}\right) \leq \frac{q(q-1)}{2^{m+1}}. \quad (128)$$

The coupling is valid for adaptive queries, and deterministic tagging and truncation introduce no additional error. With fixed independent instances $j = 0, \dots, h-1$, let q_j count **all** calls to instance j and $Q = \sum_j q_j$. Couple every function/permutation pair independently and run the distinguisher once against the product coupling. Conditional on any common transcript before the i th distinct query to instance j , the next function output collides with one of its preceding outputs with probability at most $\frac{i-1}{2^m}$. Hence disagreement is contained in the union of these per-instance collision events, and

$$\text{Adv}_{\text{parallel}} \leq \sum_j \frac{q_j(q_j-1)}{2^{m+1}} \leq \frac{Q(Q-1)}{2^{m+1}}. \quad (129)$$

The cumulative wrapper coordinates are Equation 127 with $q = Q$. Global peak space, however, cannot reuse its single-instance expression: all $2h$ logical wrapper parameter tracks coexist. A conservative sum of the h per-instance strong space grades gives

$$\text{Space}_{\text{parallel}} \leq S + h(4m + 2\kappa + 10). \quad (130)$$

This bound deliberately does not infer buffer nonoverlap across occurrences; a sharper single-token simultaneous-state analysis may replace it. For $m = 256, s = 32, n = 224, \ell = 128, h = 16$, and $q_j \leq 2^{20}$, one has $Q \leq 2^{24}$, loss below 2^{-209} , per-query wrapper work 2668, and total wrapper work at most 44,761,612,288; the conservative wrapper-space increment is $32\kappa + 16,544$. The large concrete costs beside the small statistical loss are precisely what the profile layer is meant to expose. If h varies with κ , this argument additionally requires the indexed product-state specification described above; the fixed- h statement does not supply it for free.

16.6 The efficient construction witness

The lower layer should not replace a constructive statement by the slogan “there exist PPT machines.” It supplies a checkable witness package. For a selected uniformity mode M and completion class $\mathcal{C}$, write

$$\text{EffConstruct}_{M,\mathcal{C}}(\pi R, S\sigma; T_\pi, T_\sigma, \varepsilon, \delta, \eta) \quad (131)$$

when the following data and obligations are present:

- fixed protocol and simulator codes π, σ , or fixed generators followed by the universal compiler of Section 14;
- fixed encodings, grades, ownership labels, and bounded-call contracts for every named specification occurrence;
- route-safe canonical plumbing as in Section 13, with every intentionally limited communication mechanism represented by an explicit resource node;
- polynomial profile transformers T_π, T_σ accounting for work, peak space, randomness, traffic, activations, named oracle access, and terminal-report/scorer processing;
- for every admitted uniform behavioral test, after the appropriate profile reindexing,

$$\text{Adv}_{D,p_D;M}^{\kappa(U(\pi R), U(S\sigma))} \leq \varepsilon_{D,p_D}(\kappa), \quad (132)$$

where ε_{D,p_D} is negligible for that fixed test and workload policy; a proof may instead supply the stronger concrete bound $\varepsilon(\kappa, \text{Profile}_D)$ uniformly over a profile-bounded class;

- for every $C \in \mathcal{C}$, tested-side, context-side, and shared-infrastructure no-exhaustion certificates in **both** closed experiments $C[\pi R]$ and $C[S\sigma]$, with the sum of all six owner-class bounds at most $\delta_{C(\kappa)}$ (or a displayed uniform concrete envelope);
- closed-productivity failure bounds for both experiments on the advertised workload, each obtained by adding tested-graph progress failure to the completion context's χ_C bound, whose sum is at most $\eta_{C(\kappa)}$; and
- an explicit statement of whether the claim is pure uniform, bounded-auxiliary uniform, or nonuniform advice.

In the overwhelming form, δ_C and η_C are negligible for every fixed C and workload policy. The strong form replaces the corresponding failure events by the empty event; the almost-sure form gives them probability zero without silently promoting that fact to a pathwise statement.

Arbitrary self-exhausting tests remain admissible for the behavioral inequality because Equation 68 gives them a total semantic decision. They are excluded only from the availability conclusion: a context cannot spend its own quota and then attribute the resulting owner-labeled exhaustion to the tested resource. If the security theorem is first proved for an unmetered intended execution with error ε , the coupling theorem in Section 12 changes the metered behavioral bound for C by at most δ_C : the triangle inequality uses one exhaustion coupling term for each of the real and ideal experiments. The productivity quantity η_C remains separate and is combined only when the intended statement requires a closed successful decision.

Profile transformers and negligible functions compose as in the abstract construction calculus, while owner-specific safety certificates compose by the elementary or solved response-adaptive DAG theorem, the affine-credit theorem, or another advertised progress rule of Section 12. Thus Equation 131 is evidence attached below a construction; it does not alter the higher construction relation.

16.7 The lower AC interface

At each fixed (κ, b) , the finite costed and behavioral quotients and the homomorphism

$$U : \mathcal{R}_{\kappa, b}^{\text{cost}} \rightarrow \mathcal{R}_{\kappa, b}^{\text{beh}} \quad (133)$$

have already been constructed in Section 12. They include bounded-call specifications by the finite terminal-kernel theorem, every legal finite connection, cycles that end by metering, and polynomially graded converter composition. These facts no longer depend on a random-system lifetime carrier.

On the route-safe subalgebra, Equation 77 supplies the further target map

$$J : \mathcal{R}_{\kappa, b}^{\text{beh,rs}} \rightarrow \mathcal{R}_{\text{partial}} \quad (134)$$

and proves that it is an injective system-algebra homomorphism. Consequently the route-safe operational carrier satisfies the system-algebra premises on which the ordinary Constructive-Cryptography theorem depends; $J \circ U$ identifies its erased result as a partial random system. Equation 121 supplies the computational test metric and lifts the pointwise operations to the uniform efficient witnesses above. A deliberately limited communication mechanism is not excluded; it appears as an explicit resource node and is therefore retained by J . Only comparison with a separately imposed target carrier using a different nonresponse or feedback convention remains conditional. This is a lower realization of the AC hierarchy, not a translation of Universal Composability.

Theorem Efficient lower cryptographic algebra. Fix effective boundary codecs, the pure-uniform, bounded-auxiliary, or explicit-nonuniform mode, and one finite dependency signature Γ of fixed efficiently accessible specification packages. Let resources be Γ -relative route-safe uniform graded graph families—using either fixed templates or generated presentations satisfying Section 14—and let converters and behavioral tests use the codes admitted by the selected mode, with the charged terminal scorers above. In the first two modes these are fixed uniform codes; in the last, the advice/circuit family and its polynomial bound are explicit data.

Then typed renaming, fixed finite independent parallel composition, every legal finite connection, identities, and converter action are closed in this class. Behavioral computational indistinguishability is an equivalence

relation compatible with those operations after their displayed polynomial profile reindexing. Sequential construction adds negligible errors and composes profile transformers. Hence these objects supply the resource, converter, and distinguisher data needed to instantiate the ordinary Constructive-Cryptography construction theorem.

Proof. At each (κ, b) , Equation 77 supplies the system operations and their order laws. Fixed templates are closed by finite graph union and connection; the generated presentation is first compiled to a fixed template. Recomputing Equation 72 uses only finite sums and compositions of the fixed grades, so it remains polynomial and introduces no parameter-dependent code.

Identity is a bare typed alias. Sequential and fixed finite parallel converter composition combine fixed codes and compose or add their profile transformers. Parallel composition first alpha-renames occurrence namespaces and then retains every dependency's kernel, initialization coordinate, seed coordinate, tariff, and independence convention. Exact converter and parallel-resource absorption identifies the corresponding closed experiments pathwise; Equation 121 and its T_S analogue move the absorbed test to the reindexed profile. Polynomial substitution preserves the test class. Reflexivity and symmetry are pointwise, while transitivity is the triangle inequality followed by closure of negligible functions under finite sums. The same triangle argument composes construction errors. $\square$

In the notation of Abstract Cryptography, take Φ^f to be the admitted route-safe families relative to Γ , Σ^f the admitted converter families, and $\mathcal{D}_\Gamma^f$ all admitted charged-scorer test networks relative to that same signature. This is a class of networks, not merely bare terminal-scoring programs: it permits any fixed finite feasible graph around the challenge. Converter actions at distinct interfaces commute by connection-order invariance; the bare alias is neutral; computational equivalence is a congruence; and the two absorption cases close $\mathcal{D}_\Gamma^f$ under emulating every converter in Σ^f and every parallel resource in Φ^f . These are precisely the cryptographic-algebra and compatible-distinguisher closure clauses of [1, Defs. 14, 16-17], now proved relative to the declared lower dependencies. One may select an honest efficient converter subclass $\Sigma^e \subseteq \Sigma^f$ by a stricter grade policy, provided it is closed under serial composition; the neutral identity already belongs to Σ^f and may also be included in the honest class. Taking all fixed polynomial grades for both classes is the ordinary special case.

This theorem is about security closure of bounded lower behaviors, including behaviors that exhaust. Calling a protocol an **adequate efficient implementation** additionally requires the owner-aware no-exhaustion, productivity, and realization fields of Equation 131. Those fields compose only under their DAG, rank, affine-credit, or other advertised progress certificate; they are not silently inferred from the cryptographic algebra.

16.8 Simulator cost

A simulator is an ordinary graded converter. Its work, space, randomness, traffic, and oracle calls occur in the ideal experiment profile. In a free-efficient model the profile proves that it remains in the admitted class. In a concrete theorem the profile is part of the reduction loss.

If a construction statement is intended to treat the simulator's computation as supplied functionality, one available modeling choice is to reify it on the ideal side. Maurer and Renner illustrate this choice by replacing a computational system β with a parallel behavioral resource $\bar{\beta}$ and a trivial connector [3, Sec. 4.3]. They neither mandate this choice for every concrete-security theorem nor select a Turing processor API. The processor/store/coin decomposition below is our lower refinement of their third modeling regime.

17 Explicit computation resources

17.1 Free efficient computation and charged computation

The metered model still treats every admitted polynomial grade as free from the viewpoint of a higher construction statement. It distinguishes efficient from inefficient implementations but does not yet require a construction to supply the processor or memory it consumes.

When a cost matters to the statement, the CC modeling principle moves it into the resource. Maurer and Renner distinguish unrestricted converters, stateless converters with memory, routers with a processor, and a

free efficient converter class [3, Sec. 3.5]. The exact ledger is compatible with all four choices; only one private sequential instance of the third is proved below.

Design proposal The lower-resource menu. Depending on which cost is relevant, a refinement may expose processor transitions, mutable storage, random-bit access, communication, or a clock. These names do not determine an API: random-access memory differs from a sequential tape, buffer ownership differs from copying, and a visible clock differs from a non-observable meter.

We now select one member of this menu rather than leave the entire refinement schematic. It is a sequential transition-token computer matching the machine model of Section 4. It is not asserted to be a canonical hardware model.

17.2 Statelessness and exclusivity

If all persistent memory is intended to be supplied by MEM, protocol converters must not retain hidden work-tape state. If randomness is supplied by RAND, they must not also have a private random tape. A processor separated from both resources must be only a transition engine; otherwise its hidden memory or coins bypass the accounting.

This is the modeling lesson of the Ristenpart-Shacham-Shrimpton example discussed in Section 10. Capacity alone is also insufficient to specify secure erasure, snapshots, leakage, or access patterns. Those are distinct resource features.

17.3 A selected sequential computer

Fix a bit-costful machine program P at occurrence v and a native quota

$$B_v = (T_v, A_v, R_v, S_v, \dots), \quad (135)$$

where the displayed coordinates bound committed transitions, activations, random-tape reads, and native logical live cells. The omitted coordinates are the original edge-traffic and specification-call quotas. Let L_v bound the fixed program encoding.

The selected output-register normal form is the bit-costful convention of Section 11: it is blank on activation, and a well-typed emitting configuration contains one self-delimiting code contiguously from a designated origin. A machine with an arbitrary sparse output tape must first run a charged copy/normalization routine; the refinement then applies to that transformed native ledger.

This normal form supplies a crucial amortized reach bound. For fixed P , there is λ_P such that every native prefix satisfies

$$\sum_{e \in \text{AttemptEmit}} (|\text{enc}(e)| + 1) \leq \lambda_P(1 + \text{Acts} + \text{Steps}). \quad (136)$$

The sum includes an output whose unchanged downstream router later rejects. The register is blank at activation and unaliased; a transition changes only a fixed number of cells and moves each head by at most one. Charge every reached output position since the last activation to the transition that writes it or first reaches it. Suspension gives at most one attempted output per activation, proving Equation 136.

Definition Processor-store-coin resources. The selected private resources are:

- $\text{PROC}_{v[P, T_v, A_v, L_v]}$, an **initialized, program-indexed** transition engine. Its state contains one immutable copy of $\text{code}(P)$, of length at most L_v , transition and activation tokens, and one fixed-size reservation flag. It stores no mutable machine configuration and has no random source.
- $\text{STORE}_{v[S_v]}$, which owns the finite control, mode, input, work, and output tapes, immutable parameter/auxiliary tracks, ordinary heads, and pending port. Its logical capacity test is exactly $\text{live}_{P(c)} \leq S_v$, including occupied parameter-track cells in a uniform family. It can expose only the scanned local frame, including the scanned output-register symbols but no numeric head positions, and tentatively apply one constant-locality command; it cannot evaluate P . An outstanding update capability carries a private meter-readable charging view containing only the prepared outcome tag, port class, and

exact encoded output length. On an emitting outcome only, a private read-only scan of the prepared contiguous buffer computes that length with linear access work and logarithmic scratch.

- $\text{COIN}_{v[R_v]}$, which owns the named random tape, its head, a read counter, and one reservation flag. A read returns the current bit and advances at commit iff the actual native command requests it. Repeated non-advancing reads return the same bit and remain separately charged.

Before the first activation, the private access meter dedicates a coordinatewise capacity

$$\text{OutPool}_{P(A_v, T_v)} = g_P \lambda_{P(1+A_v+T_v)} \quad (137)$$

for a fixed codec/routing constant g_P . Dedication locks capacity but commits no work or traffic. Actual emitting scans and private store-to-driver output bits draw their exact cost from this pool. Equation 136 proves that these draws cannot fail, including when the next original-edge reservation rejects. The pool counter and unused capacity are ordinary explicit state.

The program, counters, and transaction scratch are not physically invisible. The explicit report includes program storage, the logarithmic counters, a fixed transaction frame, and the live routing buffer. The selected APIs merely take their own token, capacity, and ledger operations as primitives. Refining those primitives into gates, energy, or another memory hierarchy would be the next lower layer.

The processor exposes `reserveWake`, `reserveStep`, `plan`, and infallible commit operations. The store exposes `accept.prepare`, `frame.update.prepare`, and infallible commits. The coin source exposes `reserveMode(use?)` and `commitRead(cap, "advance"?)`. Its fixed-shape mode is either a genuine non-destructive peek or a no-read ticket; the latter cannot fail and changes no coin state. The advance flag is supplied only after the processor sees the bit, because a native transition may make that decision bit-dependent. A reservation locks capacity but changes no native ledger coordinate. Capabilities are linear, and because execution has one token, each resource has at most one outstanding reservation.

Definition Explicit component translation. Let

$$E_{v(P, B_v)} = \text{Drive}_P[\text{PROC}_{v[P; T_v, A_v, L_v]} \parallel \text{STORE}_{v[S_v]} \parallel \text{COIN}_{v[R_v]}] \quad (138)$$

Drive_P is a generated finite routing template. It renames native ports and dispatches fixed transaction tags. The subscript records the native port typing; it is not a program image or a program-selection message. Connection to the already initialized, program-indexed processor selects P . The driver retains no payload, program bits, or continuation, evaluates no transition, samples no bit, and cannot inspect a quota. Every dynamic phase is in `STORE` or the unique in-flight message. After connection, only the native ports are external.

Giving another party direct access to a private processor, store, coin, or transaction port defines a shared-resource model and is not covered by this translation. Different machine occurrences receive differently named coin resources with product tape law unless sharing is represented by one explicit common resource.

Proposition Admissibility of the selected APIs. The processor, store, and coin nodes satisfy the bounded-call contract of Section 15. Their state and message spaces are standard Borel, their codecs and public reservation maps are effective, and every reply has a strong public bound. For fixed P , processor and control replies have fixed length; coin replies have one payload bit; and a store output is bounded by $c_{P(S_v+1)}$ for a fixed alphabet/codec constant c_P . After `update.prepare`, the private charging view supplies the exact commit length ℓ , so the commit reservation can use $u_P + v_P \ell$ rather than the worst-case store bound. Producing the view has tariff $u_{\text{len}} + v_{\text{len}(\ell+1)}$ and $O(\log(S_v + 1))$ scratch on an emitting outcome; internal and block outcomes do not scan the output buffer.

Proof. Finite-support tape configurations, integer heads, finite control, and bounded counters form a countable state space. The coin tape is the standard Borel space $\{0, 1\}^{\mathbb{N}}$ with a countable head coordinate; current-bit evaluation and optional head advance are measurable. Program lookup, local tape update, capacity comparison, capability validation, and every codec are effective finite operations. Fixed control calls reserve their fixed envelopes. The store's global $c_{P(S_v+1)}$ envelope proves bounded accessibility. On `Emit`, prefix-free decoding of

the prepared contiguous buffer finds its terminator in $O(\ell + 1)$ access work without copying or committing the payload; well-typedness excludes an invalid-code branch. The scan and private output passage take exact sub-capabilities from Equation 137; the prepared commit then reserves its exact remaining response/routing envelope. The reach proof funds this even if the next original router rejects and therefore commits no original traffic. Reserving $c_{P(S_v+1)}$ anew on every step could spuriously reject a late small output and is not used by the refinement theorem. The no-read coin mode returns its fixed dummy response and has no random-read charge. $\square$

17.4 Transactional simulation

A direct sequence “decrement processor, read coin, then try the store” is incorrect. If the actual update exceeds S_v , the native meter rejects the joint primitive transition before committing its step or random-read coordinate. The direct sequence has already spent both. Reserving the worst-case random branch is also incorrect because it may reject while the actual branch fits.

The translation instead uses two-phase reservation.

1. On activation, the validated self-delimiting event header gives its length while the value remains in the charged routing buffer. The store reserves the prospective input region and the processor reserves one activation; the private meter also reserves the exact remaining copy/commit cost from that length. If both primary reservations fit, the adapter copies the value once into that region and commits both reservations.
2. On every transition, the store sends the constant-size scanned frame and the processor reserves one transition. It classifies the instruction as no-read or read. The coin resource receives the same padded request shape. In the no-read case it returns a dummy bit without reserving a random-read token; otherwise it reserves a read and non-destructively returns the current bit.
3. The processor returns the native command. For a no-read instruction the dummy bit is ignored; for a read it also returns the actual advance flag. The store reserves the prospective native footprint of the actual command. On an emitting outcome only, an infallible Equation 137 sub-capability pays the actual linear scan of the prepared contiguous self-delimiting output; its charging view then gives the exact length. The meter takes the exact private output-passage sub-capability and reserves every remaining fixed commit/API charge. Only after every primary reservation succeeds does the protocol commit coin with that advance flag or its no-read acknowledgement, processor, and store in that order.
4. The final store commit is the only phase that can start the next native transition, emit the already constructed output register, or produce strict Block. Every earlier commit is private and infallible.

All primary quota failures occur before a native coordinate commits and carry the same public owner v . Linear reservation capabilities prevent replay. The derived administrative envelope makes every private tariff/routing reservation fit, so it creates no additional terminal branch; the check is external meter bookkeeping rather than another message phase. For response-dependent output work, this claim follows from Equation 136 and Equation 137, not from an assumption that the downstream traffic reservation will succeed. Fixed padded control records and the no-read ticket make the number and length of pre-commit messages independent of whether a bit is requested and of its secret value. Native output length, space, or failure may of course depend on that value exactly as it did before refinement.

Give a private state the lexicographic rank (r, n) , where $r \in \{8, 7, \dots, 1\}$ is the remaining phase suffix—frame, processor reservation, coin mode, plan, store reservation/scan, coin acknowledgement, processor commit, store commit—and n is the number of remaining microsteps in the currently admitted scan or bit copy. Each such microstep decreases n ; a phase change decreases r regardless of the next finite n . Reservations either answer or terminate with Exhaust, admitted lengths are finite, and commits cannot fail. Thus a native primitive uses at most

$$k_P + c_{P,\text{in}}L_{\text{in}} + c_{P,\text{out}}L_{\text{out}} \quad (139)$$

private microsteps through at most eight phases. The well-founded lexicographic rank, rather than the phase count alone, excludes a new administrative livelock.

17.5 Macro refinement

For a native network N , let $E(N)$ replace every machine occurrence by Equation 138, alpha-renaming private names. It retains every original typed edge, its quota, and every selected specification occurrence. Private administrative edges receive the route-safe envelope derived from Equation 135 and the fixed eight-phase protocol.

Theorem Sequential explicit-resource refinement. Let N be a well-typed metered single-token network. Fix its tuple of named tapes, selected specification seed sequences, initial specification states, and quota vector B . Then

$$\text{erase}_{\text{adm}}(E(N)_B) = N_B \quad (140)$$

as maximal owner-labeled transcript semantics. Moreover, for every corresponding finite prefix $\hat{\rho}$,

$$\pi_{\text{native}}(\text{Cost}_{E(N)}(\hat{\rho})) = \text{Cost}_{N(\rho)}. \quad (141)$$

The projection retains committed transitions, activations, random reads, native local peaks and native simultaneous global peak, original-edge traffic, and original specification calls. Program storage, physical simultaneous peak, and administrative coordinates remain in the full explicit report.

Proof. At a transaction boundary relate the two states by equality of the native configuration stored in STORE, equality of the native tape/head with COIN, equality of committed token counts with the native ledger, and absence of a reservation. On activation, the two reservations test exactly the prospective input footprint and activation coordinate. On a transition, equal frames and program code select the same instruction. In the random case equal coin tape and head give the same bit. The store therefore tests precisely the actual successor footprint.

The output-pool invariant makes the prepared scan and private output passage fit even before the original router has admitted the value. The remaining derived administrative envelope makes every other private access and routing reservation fit. The primary reservations succeed exactly when the native joint charge fits. Their infallible commits establish the state relation for the successor and add precisely the native projected cost. If a primary reservation fails, no native coordinate has committed and both sides return Exhaust with owner v . Internal, emit, and block commands agree after the last store commit.

The projected global-peak coordinate is updated only from the related native configurations at transaction boundaries and from the unchanged original routing configurations. Prepared store state, transaction messages, program storage, and counter state instead contribute to the full physical gpeak of $E(N)$. Thus their possibly larger simultaneous footprint is retained without corrupting the exact native projection.

Fold this stuttering step over the unique token run. A hidden or boundary event is exposed only after a completed transaction, so the original edge next receives the same value. The argument follows the token and therefore also covers cyclic graphs. The phase/microstep rank excludes a new infinite administrative suffix; an infinite native run expands to an infinite explicit run. This proves both displayed equalities. The result is pointwise in tapes, specification seeds, and initial states, so pushing forward their common law also gives equality of the experiment kernels. $\square$

This is deliberately a projected cost equality. A cost-aware observer allowed to score the new administrative coordinates can distinguish the presentations; a behavioral observer, or a cost observer composed with π_{native} , cannot.

Proposition Structural naturality of explicit translation. Up to alpha-isomorphism of private names,

$$E(\rho N) = \rho E(N), \quad E(N \parallel M) = E(N) \parallel E(M), \quad E(\gamma_{p,q} N) = \gamma_{p,q} E(N). \quad (142)$$

Administrative erasure preserves these equations. Hence the selected explicit-resource image is itself a lower system algebra and $\text{erase}_{\text{adm}}$ is an operation-preserving map to the metered machine algebra.

Proof. Replacement is occurrence-local. Renaming transports the generated private names, disjoint union produces disjoint resource bundles, and the translation preserves every native boundary port. Connection therefore

adds the same original matching after either construction order. The private transaction edges are unaffected and disappear under erasure. $\square$

17.6 Explicit overhead and adequate efficiency

For fixed P , finite constants $a_P, b_P, c_P, d_P, e_P, h_P$ satisfy

$$\text{AdminWork}, \text{AdminTraffic} \leq a_P + b_P \text{ Acts} + c_P \text{ Steps} + d_P \text{ Coins} + e_P \text{ OriginalTraffic}, \quad (143)$$

$$\text{AdminSpace} \leq h_P + \text{MaxEvent} + O(\log(S_v + 1)). \quad (144)$$

Control frames are fixed and padded. Here OriginalTraffic includes every original or boundary payload bit whose delivery reaches a native activation; its coefficient pays the one necessary private input passage. Output scans and private store-to-driver output passage are charged instead to the activation/step coefficients by Equation 136. This remains valid when the unchanged downstream router rejects an attempted output and commits no original traffic. The logarithmic administrative term stores its exact prepared length. The constant a_P covers the unique reserved but not yet committed local action at a prefix inside one transaction. Let K_P be the fixed finite set of counter coordinates owned by the explicit bundle and its private access meter, including transition, activation, random-read, logical space, administrative work/traffic, reservation, and output-pool counters. If B_v^{exp} is the native budget extended by the derived administrative envelope, put

$$\text{CtrBits}_{P(B_v^{\text{exp}})} = \sum_{x \in K_P} \lceil \log_2(B_{v,x}^{\text{exp}} + 1) \rceil. \quad (145)$$

Used and remaining values change this expression by at most a fixed factor. Original-edge and named-specification meters left outside the bundle remain serialized once in the surrounding graph and are not duplicated. The full explicit space also includes

$$L_v + \text{CtrBits}_{P(B_v^{\text{exp}})} + O((S_v + H_P) \log(T_v + S_v + 2)). \quad (146)$$

under the canonical sparse serialization used for explicit terminal reports. Indeed, there are at most S_v live cells and H_P ordinary heads. From a standard initialized configuration, every such coordinate has magnitude at most $T_v + S_v + 1$: heads start at designated origins, inputs are installed contiguously, and a transition moves a head by at most one. A fixed tape alphabet and fixed number of tapes therefore give the displayed bound for cell labels, symbols, and heads. The explicit counter sum covers every resource and access-meter coordinate, not only the four native coordinates shown in Equation 135. The native logical capacity remains exactly S_v ; the named projection intentionally forgets this representation overhead. The random-tape coordinate is already covered by the random counter in CtrBits_P , while the infinite sampled tape is not serialized. The L_v term counts the processor's unique immutable program copy; no copy is hidden in Drive_P .

For completeness, this local statement yields a network-wide physical peak without attempting to reconstruct simultaneity from local maxima. In a fixed closed route-safe experiment, let E_N dominate every original or private in-flight event, and let I_N dominate the serialized state and counters of the fixed routing, staging, and shared-meter infrastructure. A conservative simultaneous bound is

$$\begin{aligned} \text{PhysGPeak}(E(N)) \leq I_N + \sum_{v \in N} [& \\ & L_v + \text{CtrBits}_{P_v}(B_v^{\text{exp}}) \\ & + O((S_v + H_{P_v}) \log(T_v + S_v + 2)) + h_{P_v} + E_N]. \end{aligned} \quad (147)$$

The sum deliberately overcounts the single live token and its message. It is nevertheless a sound bound on every physical configuration, including coexisting persistent program, counter, and store states and the one active transaction. Since the normalized graph is fixed and all terms are fixed polynomials after the closing context chooses its workload, the right side is polynomial. The **exact** physical gpeak is still updated from actual configurations; Equation 147 is only its funding certificate.

A generic loaded presentation is a different construction, not alternate notation for Equation 138. It needs a charged boundary program event or an explicit $\text{IMAGE}_{v[P, L_v]}$ source and an initially blank processor. A copying loader must reserve the destination before commit, validate the self-delimiting code, and pay linear

program traffic and work, $O(\log(L_v + 1))$ scratch, and the simultaneous source/destination peak. A one-shot ownership transfer or erasing image source would be a different resource API. Once a valid load is fully funded and committed before the first native activation, the initialized theorem applies to the residual run; without such a source and tariff there is no loader theorem.

Proposition Efficient explicit refinement. A pure uniform polynomially graded native family translates to one polynomially graded explicit-resource family. For a secondary generated network, first apply the generated-to-fixed theorem of Section 14, obtaining one fixed universal machine with its **physical** polynomial profile, and then translate that fixed occurrence by Equation 138. The composite processor/store/coin and administrative profiles are polynomial.

Directly replacing every decoded node by a fresh explicit resource bundle is not admitted: it would manufacture a parameter-dependent family of specification-resource occurrences. Such a presentation needs a separately defined indexed product resource and aggregate tariff.

If the native family has no-exhaustion and productivity certificates, the translated family inherits them with no additional probabilistic failure once Equation 143 and Equation 144 are funded.

Proof. In the primary presentation, substitute the native polynomial profiles in Equation 143 and Equation 144. Polynomial closure gives all explicit grades. In the generated presentation, first substitute the aggregate native profile into Equation 80 and Equation 88; then substitute that physical universal-machine profile into the same explicit bounds. Fixed polynomial composition remains polynomial. No oracle/resource occurrence is generated. The phase/microstep rank proves that the translation cannot block or livelock between native steps. Then Equation 140 transfers native success, exhaustion, and realization events pathwise. $\square$

17.7 Transporting AC experiments

For a native program bundle, write

$$\mathcal{C}_{P(B_P)} = \text{PROC}_{P[P;T_P,A_P,L_P]} \parallel \text{STORE}_{P[S_P]} \parallel \text{COIN}_{P[R_P]}. \quad (148)$$

At the explicit level the former program converter is replaced by the routing-only Drive_P , while $\mathcal{C}_P$ is an assumed parallel resource. This is the concrete instance of the third modeling regime described in [3, Sec. 3.5]; the transaction API and theorem are ours.

Theorem Transport of lower construction experiments. Let X, Y be two admitted metered experiments, and select any finite set of their machine occurrences for explicit translation. Every behavioral test on the unchanged external ports has exactly the same advantage between $E(X), E(Y)$ as between X, Y . The same holds for a cost-aware test whose report factors through π_{native} . Consequently an ε -construction or simulation equation is transported with the same error and with only the polynomial profile enlargement of Equation 143 and Equation 144.

Proof. Apply Equation 140 separately to both closed tests. The terminal transcript kernels are equal pathwise; Equation 141 gives the projected-report statement. Taking the difference of equal acceptance probabilities adds no error. Uniformity and admissibility of the translated test graphs follow from the efficient-refinement proposition. $\square$

For example, a native simulation equation

$$\pi R \approx_\varepsilon S \sigma \quad (149)$$

becomes, when both program costs are reified,

$$\text{Drive}_{\pi[R|\mathcal{C}_\pi]} \approx \text{Drive}_{\sigma[S|\mathcal{C}_\sigma]}. \quad (150)$$

The interfaces of $\mathcal{C}_\pi$ and $\mathcal{C}_\sigma$ must be placed at the parties which supply those costs. Reifying only honest computation while retaining a free-efficient simulator is another declared mixed regime. If simulator cost matters, Equation 150 keeps it on the ideal side instead of hiding it inside the word “converter.” This is analogous to the $[S, \bar{\beta}] \sigma$ refactoring in [3, Sec. 4.3], but stronger in a different direction: Equation 140 proves that the particular

behavioral simulator resource is realized by the selected processor/store/coin API. That realization theorem is not supplied by MR16.

Proposition Whole-experiment computation reification. Fix one admitted closed computational test experiment in any selected input mode. Make its initialization generator, interaction components, effective terminal-state and report projections, terminal-record builder, and scorer explicit as native machine occurrences with their already declared grades. In explicit nonuniform mode, use one fixed universal circuit/advice evaluator and supply the named polynomial-length descriptions as input. For secondary generated syntax, first apply Section 14.

Replacing **all** those occurrences by their $\text{Drive}_{P[C_P]}$ presentations preserves the final decision law exactly and preserves the complete native ledger, including gpeak, after the named projection. The full processor/store/coin, initialization, administrative, and terminal-processing profile is polynomial. Thus every computational actor in a fixed closed experiment can be presented with transition evaluation, mutable state, and coin access inside explicit resources; the remaining converters perform only the declared finite tag routing.

Proof. The input generator and every effective postprocessor are fixed graded codes, so they are ordinary additional machine occurrences in the lower evaluation harness. The semantic supervisor freezes the terminal record and activates the postprocessor; this does not turn Block or Exhaust into a wire response. Apply Equation 140 and Equation 141 to every occurrence, including the postprocessor, and compose the finitely many pathwise equalities. The fixed universal evaluator handles a named polynomial nonuniform description with the same interpreter argument as Section 14. Finite sums and polynomial substitution bound the full profile. A scorer-side exhaustion still selects the same fixed default bit. $\square$

This proposition does not end the abstraction hierarchy. The lower semantic supervisor that detects a maximal run, the non-observable meters and their reservation primitives, and the chosen routing API remain primitives of this explicit level. Reifying their gates, energy, clocks, or physical memory is a further model. Moreover, a security comparison must place identical test-owned computer resources in both experiments and couple their initial states/coins as prescribed. Translating only protocol or simulator occurrences while leaving test computation in the feasible converter class is a legitimate but explicitly **mixed** regime. Changing which party is supplied which computer resource changes the resource specification and cannot be hidden inside the metric.

17.8 A selected communication resource

The label $\text{COMM}[c]$ is still too coarse for a theorem. In this sequential model select a one-place lossless link

$$\text{SLINK}_{e[C,W,S]}. \quad (151)$$

On a validated code of length ℓ , it atomically checks

$$\text{usedBits} + \ell \leq C, \quad \text{usedWork} + a_0 + a_1 \ell \leq W, \quad \text{buffer}(\ell) \leq S. \quad (152)$$

If they fit, it performs the canonical bit copy, clears the old buffer, and delivers exactly one equal event. Otherwise it returns $\text{Exhaust}(e)$ without delivery. It has no clock and cannot drop, duplicate, reorder, or leak a message.

Proposition Sequential-link refinement. Replacing a metered canonical edge by SLINK_e with the same traffic, router-work, and buffer quotas preserves its maximal transcript and those ledger coordinates, up to the declared infrastructure-owner renaming.

Proof. Relate the unique routing buffer and remaining capacities. The self-delimiting length is known before allocation. Both presentations first make the same owner- e atomic reservation. On rejection neither commits a routing coordinate or buffer. On admission both execute the same infallible bit-costful copy, with the same intermediate buffer occupancy, transition, traffic, and gpeak updates, and then deliver the identical event. $\square$

When its three bounds dominate Equation 72, SLINK is transparent. With smaller bounds it is an intentional fallible node. Asynchronous queues, contention, packetization, adversarial delivery, shared processor access, reset, leakage, and secure erasure are different lower APIs.

A visible CLOCK is not a conservative refinement of the present carrier: a context could distinguish two equal first-visible-output behaviors by their number of private phases. Time must either remain a hidden meter, be retained in a timed target, or be paired with an explicitly clock-insensitive test class.

The refinement hierarchy is therefore:

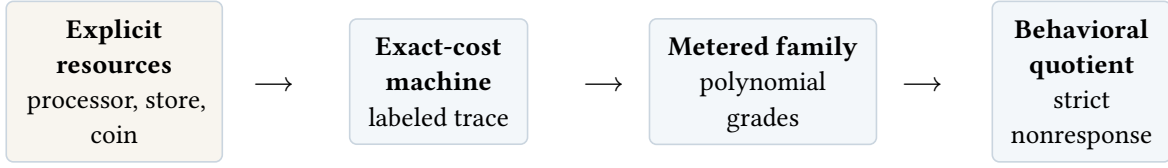


Table 1: Each rightward map forgets a selected implementation distinction.

For the selected sequential APIs, the first arrow is Equation 140 together with the native cost projection Equation 141. It remains a design obligation for a different RAM, shared processor, link, leakage, or timed API. The next arrow selects and enforces a grade. The last is the proved cost erasure into the finite behavioral operational algebra. The additional map from that algebra to the selected partial-random-system carrier is the route-safe homomorphism J in Equation 134. Only comparison with another carrier using different nonresponse or feedback remains conditional.

18 Beyond the token: a causal projective alternative

18.1 What the single-token model omits

The token model has one external input followed by one first visible output. It is a good match for query-response random systems and for converter application. It does not directly express:

- two unordered pending messages;
- spontaneous outputs before an external query;
- a scheduler whose choices are observable or adversarial;
- simultaneous events, relativistic constraints, or explicit time;
- a system whose total behavior contains an unbounded stream of events not representable as one finite global value.

Encoding a scheduler as an explicit machine handles many sequential uses. It does not faithfully turn genuinely unordered events into an alternating history without adding an order that was absent from the phenomenon.

18.2 Consistent behavior on finite causal cuts

A more general classical model can be organized as follows. Choose a partially ordered set T of event positions. A boundary event records a port, direction, payload, and position $t \in T$. A cut $C \subseteq T$ is downward closed; it represents the part of a run observed up to a causally closed frontier.

Instead of one global transition on a possibly infinite event history, specify a family of probability kernels

$$(K_C)_{C \in \mathcal{C}_b} \quad (153)$$

on all bounded finite or suitably finite causal cuts. Require:

- **Consistency:** if $C \subseteq D$, restricting the output law K_D to C gives K_C .
- **Causality:** output on C depends only on input on a strictly earlier cut $\chi(C)$.
- **Finite causal descent:** iterating χ removes every bounded position in finitely many steps, excluding Zeno feedback before a finite frontier.
- **Local finiteness:** every bounded observation contains only finitely much event data, or is represented by a finite-dimensional cylinder.

Composition synchronizes events at connected ports, hides them, and restricts the resulting family to the remaining boundary. Parallel composition uses product kernels when the component randomness is independent.

This pattern is motivated by causal boxes. They use mutually consistent maps on bounded cuts so a system can have unbounded total message behavior without requiring a single infinite tensor-product output [14, Secs. 4.1-4.3]. Their causality function includes a finite-step condition specifically to exclude infinitely many feedback events before a bounded position.

18.3 Why projective semantics helps

The projective family in Equation 153 contains every finite observation while declining to package an infinite global trace prematurely. It handles an unbounded stream in the same spirit that finite transcript laws handle an infinite-tape random system. Consistency provides the extension data needed to compare larger and smaller observations.

It also addresses a closure difficulty identified for sequence-domain system algebras. Allowing “arbitrarily many, but always finitely many” messages is not closed under the ordinary omega-chain supremum: the supremum of prefixes of length n is infinite [5, Sec. 5.4]. A cut-indexed or projective model does not pretend that the entire infinite behavior is itself one finite sequence.

18.4 Why it is deferred

The causal model needs a substantial theory of event positions, cuts, restriction kernels, projective extension, and non-Zeno feedback. It also does not land directly in an alternating DDS. To use the current random-system layer, one must wrap a causal system with a polling or call-return discipline and prove that the wrapper exposes one answer per query.

For the initial operational realization, this extra generality would obscure the main boundary. The token model already captures unbounded sequential computation, persistent state, internal calls, recursion, and strict partiality, and it maps directly to the existing DDS interface. The causal model is therefore a principled extension path rather than a competing first definition.

19 Related work

Abstract Cryptography explicitly advocates proceeding from a general system algebra down through discrete systems to implementations, adding only the structure relevant at each level [1]. Constructive Cryptography applies that discipline to resources and converters [2]. The present paper occupies one of the lower levels anticipated there: it does not modify the construction notion.

Maurer’s random automata and random systems separate an explicit stateful random experiment from its conditional input-output behavior [7]. Lanzenberger and Maurer sharpen this separation by representing a probabilistic system as a distribution over DDSs and a random system as a transcript-behavior class [6]. Our fixed-random-tape map realizes the lower endpoint of that account and extends it from finite tables to measure-level laws generated by interactive machines. Maximal strict transcripts and the route-safe feedback theorem are new here; they are not attributed to the finite common-domain theory.

The algebraic theory of systems isolates parallel composition, connection, and composition-order invariance and proves that the latter is not automatic [5]. The network-graph presentation used here makes structural order-independence transparent; the deterministic connection correspondence remains the semantic content. Causal boxes provide a substantially richer closed model for partially ordered, potentially quantum messages [14]. Section 18 extracts only the classical projective lesson.

The abstract problem-and-reduction treatment in [4] motivates separating unconditional solver quantification from an eventual cost model. Jost’s thesis develops converter, asymptotic, memory, and randomness refinements within the same broad hierarchy [8]. Maurer and Renner’s modeling taxonomy, including their discussion of the Ristenpart-Shacham-Shrimpton example, is the direct basis for the memory discipline in Section 10 [3].

Categorical accounts formulate composable cryptography using monoidal structure and attack models [15]. Their work reinforces the value of a homomorphic lower realization, but our construction does not depend on a particular categorical security syntax.

Bottom-up frameworks such as Universal Composability begin from interactive machine execution [16]. Reactive Simulatability uses probabilistic I/O automata and distributed scheduling, and emphasizes that transition-function complexity alone is not compositional [17]. Hofheinz, Mueller-Quade, and Unruh develop continuously polynomial and polynomial-shape conditions [10]; their later work gives reactive and uniform reactive polynomial-time notions [11]. The IITM model gives a particularly detailed treatment of environment-relative runtime and assumption-conditional closure failures [12]. We use these works as adversarial evidence that a machine layer must settle activation, routing, message volume, feedback, and runtime quantifiers.

This paper does not translate any of those security definitions and imports none of their roles, corruption rules, session conventions, or simulator quantifiers. Its ambient-workload grade is a lower implementation device selected solely to obtain a closed bounded algebra beneath random systems.

20 Conclusion and next obligations

The top-down character of Constructive Cryptography leaves room for, rather than rules out, concrete computation models. The first lower layer is an unbounded operational realization. Ordinary interactive Turing machines perform any finite amount of local work, retain arbitrary persistent state, and communicate through explicit typed wires. Fixing their named random tapes gives deterministic lifetime behavior. Collapsing each external interaction at the first visible output gives a prefix-closed DDS. Pushing the tape law forward and quotienting by transcripts gives a random system.

This organization makes the abstraction boundary explicit. Microsteps, tape layout, and work are forgotten by macro-collapse. An explicit state space is forgotten by the history representation. Counterfactual couplings invisible to one execution are forgotten by the transcript quotient. At no point is nonresponse replaced by a visible error.

The efficient refinement keeps the same run and records what macro-collapse would erase. Its exact ledger separates work, peak space, randomness, traffic, activations, and ideal-resource calls. Uniform implementations use one fixed code and unary parameter. Component grades are polynomial in a shared ambient workload selected by each fixed efficient context. Non-observable meters make arbitrary finite wiring bounded, including bad cycles, while stabilization shows that every sufficiently funded finite success agrees with the unbounded run.

This closure is intentionally partial. A meter can stop a loop, but it cannot prove that a service answers. Adequate responsiveness requires a flow, rank, or credit theorem for the composed graph. Nor does efficient access make an ideal object implementable: abstract specifications are charged oracles, and their calls, messages, and tariffs enter the reduction profile.

Contextual cost equivalence and behavioral equivalence are congruences for all finite graph constructors, even on metered cycles, and cost erasure is a homomorphism between their quotients. A physical structural router must not be silently identified with a free wire if it can exhaust. The route-safe subalgebra therefore derives a dominating router envelope from aggregate grades; intentionally limited communication is an explicit resource. Maximal strict transcripts then make partial-DDS laws a standard-Borel random-system algebra, environment lifting proves feedback congruence, and finite-cylinder compilation proves pointwise full abstraction. The resulting J is an injective homomorphism. The machine-only image is deliberately proper because abstract systems may be noncomputable or use nonrealizable real probabilities. The oracle-relative carrier may assume named abstract specifications with such behavior, but that is an accessibility statement, not an implementation or surjectivity theorem.

At the cost layer, converter absorption is pathwise equality of normalized graphs. What changes is the certified resource profile. Polynomial profile reindexing yields ordinary asymptotic nonexpansion, while concrete security retains simulator and reduction overhead. Bounded-call specifications use pre-sample reservation, so admission cannot select a cheap response; outputs without a finite strong envelope require either a visible cap or a charged streaming interface. The efficient construction witness joins these ingredients without modifying the higher AC relation.

One more downward step is now concrete. The selected PROC/STORE/COIN bundle externalizes transition tokens, mutable native configuration, and named random-tape access behind a stateless driver. Two-phase reservation is essential: it preserves the ledger and owner label when one coordinate of a joint native step fails.

Administrative erasure is pathwise, the native exact ledger is recovered by projection, and all newly exposed work, traffic, program storage, counters, and scratch have polynomial bounds. A well-founded private phase/microstep rank transfers productivity. The corresponding SLINK theorem reifies a one-buffer lossless edge. This is one proved lower API, not a claim that tape storage is RAM or that hidden meters are visible clocks.

The finite common-domain PDS carrier embeds in the selected target. Comparing it with another independently mandated carrier that makes a different choice about nonresponse or feedback is a separate carrier theorem. The remaining useful extensions are:

1. **Projective comparison.** Relate maximal strict transcript laws to a compatible projective family on finite cuts for any external carrier that requires that presentation.
2. **Invariant synthesis.** Develop useful automatic or application-specific methods for finding the polynomial post-fixed points required by the proved response-adaptive DAG constructor. Existence is intentionally a certificate, not a decidability claim.
3. **Further streaming instantiations.** For each additional oracle API with unbounded outputs, choose its chunk and handle discipline and prove its own tail bound including calls, traffic, work, output work, and buffer space; the geometric-string instance supplies one complete model calculation.
4. **Further explicit resources.** When an application requires RAM, a shared processor or coin source, reset, secure erasure, leakage, asynchronous communication, or visible time, select that API and prove its own administrative or timed refinement theorem.
5. **Richer flow certificates.** Add guarded feedback and polynomial traffic fixed points alongside the proved DAG and affine-credit rules.

General concurrency and causal event structures should be treated as a separate extension, not hidden inside the token model. Likewise, a machine formalization and literal transition-table constants may eventually verify or sharpen these claims, but neither is part of the current theory contribution.

References

- [1] U. Maurer and R. Renner, “Abstract Cryptography,” in *Proceedings of the Second Symposium on Innovations in Computer Science (ICS 2011)*, B. Chazelle, Ed., Tsinghua University Press, 2011, pp. 1–21.
- [2] U. Maurer, “Constructive Cryptography: A New Paradigm for Security Definitions and Proofs,” in *Theory of Security and Applications*, S. Mödersheim and C. Palamidessi, Eds., in Lecture Notes in Computer Science, vol. 6993. Springer, 2012, pp. 33–56. doi: [10.1007/978-3-642-27375-9_3](https://doi.org/10.1007/978-3-642-27375-9_3).
- [3] U. Maurer and R. Renner, “From Indifferentiability to Constructive Cryptography (and Back),” in *Theory of Cryptography Conference (TCC 2016-B), Part I*, M. Hirt and A. Smith, Eds., in Lecture Notes in Computer Science, vol. 9985. Springer, 2016, pp. 3–24. doi: [10.1007/978-3-662-53641-4_1](https://doi.org/10.1007/978-3-662-53641-4_1).
- [4] U. Maurer, “Cryptography Foundations,” ETH Zurich, Course notes, 2018.
- [5] C. Matt, U. Maurer, C. Portmann, R. Renner, and B. Tackmann, “Toward an Algebraic Theory of Systems,” *Theoretical Computer Science*, vol. 747, pp. 1–25, 2018, doi: [10.1016/j.tcs.2018.06.001](https://doi.org/10.1016/j.tcs.2018.06.001).
- [6] D. Lanzenberger and U. Maurer, “Coupling of Random Systems,” in *Theory of Cryptography Conference (TCC 2020), Part III*, in Lecture Notes in Computer Science, vol. 12552. Springer, 2020, pp. 207–240. doi: [10.1007/978-3-030-64381-2_8](https://doi.org/10.1007/978-3-030-64381-2_8).
- [7] U. Maurer, “Indistinguishability of Random Systems,” in *Advances in Cryptology - EUROCRYPT 2002*, L. R. Knudsen, Ed., in Lecture Notes in Computer Science, vol. 2332. Springer, 2002, pp. 110–132. doi: [10.1007/3-540-46035-7_8](https://doi.org/10.1007/3-540-46035-7_8).
- [8] D. Jost, “On Generalizations of Composable Security,” Doctoral dissertation, ETH Zurich, 2020. doi: [10.3929/ethz-b-000417544](https://doi.org/10.3929/ethz-b-000417544).
- [9] T. Ristenpart, H. Shacham, and T. Shrimpton, “Careful with Composition: Limitations of the Indifferentiability Framework,” in *Advances in Cryptology - EUROCRYPT 2011*, K. G. Paterson, Ed., in Lecture Notes in Computer Science, vol. 6632. Springer, 2011, pp. 487–506. doi: [10.1007/978-3-642-20465-4_27](https://doi.org/10.1007/978-3-642-20465-4_27).

- [10] D. Hofheinz, J. Müller-Quade, and D. Unruh, “Polynomial Runtime in Simulatability Definitions,” *Journal of Computer Security*, vol. 17, no. 5, pp. 703–735, 2009, doi: [10.3233/JCS-2009-0354](https://doi.org/10.3233/JCS-2009-0354).
- [11] D. Hofheinz, D. Unruh, and J. Müller-Quade, “Polynomial Runtime and Composability,” *Journal of Cryptology*, vol. 26, no. 3, pp. 375–441, 2013, doi: [10.1007/s00145-012-9127-4](https://doi.org/10.1007/s00145-012-9127-4).
- [12] R. Küsters, M. Tuengerthal, and D. Rausch, “The IITM Model: A Simple and Expressive Model for Universal Composability,” *Journal of Cryptology*, vol. 33, pp. 1461–1584, 2020, doi: [10.1007/s00145-020-09352-1](https://doi.org/10.1007/s00145-020-09352-1).
- [13] O. Kallenberg, *Foundations of Modern Probability*, 2nd ed. in Probability and Its Applications. New York: Springer, 2002. doi: [10.1007/978-1-4757-4015-8](https://doi.org/10.1007/978-1-4757-4015-8).
- [14] C. Portmann, C. Matt, U. Maurer, R. Renner, and B. Tackmann, “Causal Boxes: Quantum Information-Processing Systems Closed under Composition,” *IEEE Transactions on Information Theory*, vol. 63, no. 5, pp. 3277–3305, 2017, doi: [10.1109/TIT.2017.2676805](https://doi.org/10.1109/TIT.2017.2676805).
- [15] A. Broadbent and M. Karvonen, “Categorical Composable Cryptography,” in *Foundations of Software Science and Computation Structures (FoSSaCS 2022)*, in Lecture Notes in Computer Science, vol. 13242. Springer, 2022, pp. 161–183. doi: [10.1007/978-3-030-99253-8_9](https://doi.org/10.1007/978-3-030-99253-8_9).
- [16] R. Canetti, “Universally Composable Security: A New Paradigm for Cryptographic Protocols,” in *Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science*, IEEE Computer Society, 2001, pp. 136–145. doi: [10.1109/SFCS.2001.959888](https://doi.org/10.1109/SFCS.2001.959888).
- [17] M. Backes, B. Pfitzmann, and M. Waidner, “The Reactive Simulatability (RSIM) Framework for Asynchronous Systems,” *Information and Computation*, vol. 205, no. 12, pp. 1685–1720, 2007, doi: [10.1016/j.ic.2007.05.002](https://doi.org/10.1016/j.ic.2007.05.002).